

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC



BÁO CÁO NHÓM 5

Chủ Đề:

BIG DATA, NOSQL & APACHE SPARK

Sinh viên thực hiện	Mã sinh viên
Vũ Tiến Đạt	23000111
Nguyễn Thị Hiền Trang	23000161
Nguyễn Văn Tuấn Anh	23000090
Vũ Đức Duy	23000107

Hà Nội, 5 - 2027

Mục lục

1	Giới thiệu chung	1
1.1	Lời mở đầu	1
1.2	Tóm tắt	2
2	Big Data	4
2.1	Định nghĩa	4
2.2	Đặc trưng	5
2.3	Các phương pháp xử lý dữ liệu phổ biến	7
2.3.1	Batch processing	8
2.3.2	Stream processing (Real - time processing)	10
2.3.3	Hyper processing	11
2.3.4	So sánh các phương thức	12
2.4	Ứng dụng của Big Data	12
2.5	Tương lai và thách thức trong việc phát triển Big Data	15
2.5.1	Tương lai của Big Data	15
2.5.2	Thách thức trong việc phát triển Big Data	16
2.6	Kết luận	16
3	NoSQL	18
3.1	Kỷ nguyên BigData và các vấn đề của RDBMS	18
3.2	Sơ lược về NoSQL	20
3.3	Các loại NoSQL	20
3.3.1	Key-value store	21
3.3.2	Document store	21
3.3.3	Column-Family Store	22
3.3.4	Graph Database	23

3.4	So sánh NoSQL với SQL	24
3.4.1	Tổng quan về SQL và NoSQL	24
3.4.2	So sánh sự khác nhau giữa SQL và NoSQL	25
3.5	Kết luận	27
4	Apache Hadoop	28
4.1	Giới thiệu về Apache Hadoop	28
4.2	Kiến trúc và thành phần chính của Apache Hadoop	29
4.2.1	HDFS (Hadoop Distributes File System - Strorage)	29
4.2.2	Map Reduce - Processing	31
4.2.3	YARN (Yet Another Resource Negotiator) - Resource Management	32
4.3	Ứng dụng	33
4.4	Hạn chế của Apache Hadoop	34
5	Apache Spark	35
5.1	Đặt vấn đề	35
5.2	Apache Spark là gì	35
5.2.1	Sự ra đời	35
5.2.2	Apache Spark	37
5.3	Kiến trúc của Apache Spark và cơ chế xử lý dữ liệu	38
5.3.1	Kiến trúc của Apache Spark	38
5.3.2	Cơ chế xử lý dữ liệu	39
5.4	Hệ sinh thái tích hợp của Spark	42
5.4.1	DDR	43
5.4.2	Spark SQL	49
5.4.3	Spark Streaming	52
5.4.4	Apache Spark MLlib	53
5.5	Ưu điểm và nhược điểm của Apache Spark:	56

5.5.1	Sự khác biệt giữa Apache Hadoop và Apache Spark	57
5.6	Ứng dụng của Apache Spark	58
5.7	Kết luận	59
6	Kết luận và định hướng phát triển	60
6.1	Tóm tắt lại nội dung đã trình bày	60
6.2	Hướng phát triển trong tương lai	60
7	Ứng dụng thực tế	62
7.1	Dữ liệu	62
7.2	Xử lý cơ sở dữ liệu với PySpark	63
7.2.1	Tiền xử lý dữ liệu:	64
7.3	Mô hình dự đoán đơn giản bằng Spark MLlib	66
7.3.1	Mục tiêu mô hình:	66
7.3.2	Quá trình xử lý dữ liệu:	66
7.3.3	Xây dựng mô hình hồi quy tuyến tính	67
7.3.4	Kết quả mô hình	67
7.3.5	Đánh giá và kết luận	70
8	Tài liệu tham khảo	71
	Tài liệu tham khảo	71

Danh sách hình vẽ

1	Ứng dụng Google Maps sử dụng Big Data để tối ưu hóa giao thông. . .	5
2	Mô hình 3V.	5
3	Mô hình 5V.	6
4	Mô hình MapReduce trong bài toán đếm số từ.	10

5	Ứng dụng Trace Together của Singapore sử dụng BigData truy vết người mắc bệnh trong đại dịch Covid-19.	13
6	Phân bố học lực học sinh được tổng hợp tự động bởi hệ thống VnEdu, áp dụng Big Data và AI.	14
7	Các loại hình NoSQL Databse.	20
8	Ví dụ về Key-value store.	21
9	Ví dụ về Document store	21
10	Ví dụ về Column-Family store	22
11	Ví dụ về Graph Database	23
12	Hadoop.	28
13	HDFS Architecture.	30
14	MapReduce.	31
15	Jobtracker And Tasktracker.	32
16	YARN.	33
17	Xử lý chậm.	34
18	Lịch sử ra đời của Apache Spark.	36
19	Các doanh nghiệp sử dụng Apache Spark.	36
20	Apache Spark.	37
21	Kiến trúc của Apache Spark.	38
22	Cơ chế xử lý dữ liệu.	40
23	Biểu diễn dữ liệu trong RDD.	41
24	DAG.	41
25	Hệ sinh thái của Apache Spark.	42
26	Cơ chế lưu trữ đệm (Caching) của RDD trong Spark.	44
27	RDD chia nhỏ dữ liệu để xử lý song song.	45
28	Quy trình hoạt động của RDD trong Apache Spark Transformations và Actions.	47
29	Narrow Transformations và Wide Transformations.	48

30	Cấu trúc và phân vùng của DataFrame trong Apache Spark.	50
31	Kiến trúc của DataSet.	51
32	Apache Spark Streaming.	53
33	PipelineModel.	54
34	Sự khác biệt giữa Hadoop và Spark.	57
35	Schema.	64
36	Dữ liệu chứa giá trị bất thường-1.	64
37	Dữ liệu chứa giá trị bất thường-2.	64
38	Doanh thu theo tháng năm 2023 và 2024.	65
39	Data sau khi lọc dữ liệu theo tháng dựa trên các năm.	65
40	Số chuyến đi dài hơn 10Km theo tháng trong năm 2024.	66
41	Kết quả của mô hình.	67
42	Biểu đồ tổng doanh thu theo từng tháng trong năm 2024, dựa trên số lượng chuyến đi.	68
43	Biểu đồ giá cước trung bình giữa ngày nghỉ lễ và ngày thường.	69
44	Biểu đồ giá cước trung bình giữa ngày nghỉ lễ và ngày thường năm 2023.	69
45	Biểu đồ giá cước trung bình giữa ngày nghỉ lễ và ngày thường năm 2024.	70

Danh sách bảng

1	So sánh các phương thức xử lý dữ liệu	12
2	So sánh sự khác nhau giữa SQL và NoSQL	26
3	So sánh Hadoop với Spark	58
4	Data Schema of the NYC Taxi and Limousine Commission Dataset	63

1 Giới thiệu chung

1.1 Lời mở đầu

Trong những thập kỷ gần đây, chúng ta đã chứng kiến một thời kỳ phát triển công nghệ phi thường, thời điểm mà những đổi mới công nghệ không chỉ định hình lại các ngành công nghiệp và cấu trúc xã hội mà còn quyết định vai trò sống còn của nguồn nhân lực chất lượng cao. Big Data được dự đoán là một trong những xu hướng công nghệ chủ đạo, đóng vai trò quan trọng trong việc tự động hóa, hỗ trợ con người đưa ra các quyết định và nâng cao khả năng sáng tạo.

Lịch sử của Big Data có thể bắt nguồn từ những ngày đầu của máy tính, nơi nền tảng để xử lý các bộ dữ liệu lớn đã được đặt ra. Ngay từ những năm 1940 và 1950, khái niệm lưu trữ và phân tích một lượng lớn dữ liệu bắt đầu hình thành. Trong những năm 1960 và 1970, máy tính lớn xuất hiện, cung cấp khả năng lưu trữ dữ liệu lớn. Các tổ chức bắt đầu nhận ra tiềm năng của việc lưu trữ khối lượng thông tin khổng lồ, dẫn đến sự phát triển của Hệ thống quản lý cơ sở dữ liệu (DBMS) ban đầu. Những năm 1980 - 1990 chứng kiến một bước nhảy vọt đáng kể với sự ra đời của cơ sở dữ liệu quan hệ, cung cấp một cách quản lý dữ liệu linh hoạt và có thể mở rộng hơn, đánh dấu một sự thay đổi mang tính cách mạng với sự phát triển của internet và sự phổ biến của các công nghệ kỹ thuật số. Sự bùng nổ của phương tiện truyền thông xã hội, điện thoại di động và Internet vạn vật (IoT) đã tạo ra một cơn sóng dữ liệu chưa từng có. Khối lượng, tốc độ và phạm vi dữ liệu tuyệt đối được tạo ra đã dẫn đến việc đặt ra thuật ngữ "Big Data" vào khoảng những năm 90 của thế kỷ trước. Ngày nay, khối lượng dữ liệu vô cùng lớn, kích cỡ lên đến hàng trăm terabyte cho đến petabyte chỉ cho một tập hợp dữ liệu. Cùng với đó, khi mà phần lớn dữ liệu sinh ra đều ở dạng phi cấu trúc (hình ảnh, video, bài hát, dữ liệu từ các cảm biến...) thì những phương pháp lưu trữ dữ liệu truyền thống không thể đảm đương được. Những phương pháp đó không cho phép liên kết và phân tích nhiều dạng dữ liệu khác nhau. Khối lượng dữ liệu gia tăng nhanh nhưng tốc độ xử lý dữ liệu (thu thập, xử lý và phản hồi) lại mất nhiều thời gian trong khi nhu cầu của con người ngày càng đòi hỏi cao hơn, đặc biệt là về tốc độ xử lý. Điều đó thúc đẩy con người tìm đến những giải pháp mới để xử lý và quản lý dữ liệu một cách hợp lý và tối ưu nhất.

Big Data đã và đang tiếp tục trở thành một yếu tố cách mạng thay đổi cuộc chơi trong nhiều ngành: công nghiệp, sức khỏe, tài chính và ngân hàng... Tiềm năng to lớn của Big Data trong việc phân tích lượng dữ liệu khổng lồ theo thời gian thực đã mở ra những cơ hội mới, đổi mới và tăng trưởng. Bài báo cáo này sẽ cung cấp cho các độc giả một số thông tin tổng quát về Big Data, NoSQL và Apache Spark... để giải quyết các bài toán về dữ liệu, giúp độc giả hiểu rõ hơn về các khái niệm này và cách chúng có thể ảnh hưởng đến việc xử lý dữ liệu trong thế giới hiện đại. Bằng việc khám phá Big Data, NoSQL và Spark, chúng ta có thể nắm bắt cơ hội mới và khám phá những giá trị tiềm ẩn trong dữ liệu lớn, từ đó tạo ra những đóng góp to lớn cho nhiều lĩnh vực quan trọng hiện nay.

1.2 Tóm tắt

Big Data, NoSQL và Apache Spark là ba khái niệm quan trọng trong lĩnh vực xử lý dữ liệu lớn và phân tán. Sự phát triển mạnh mẽ của công nghệ thông tin và truyền thông đã dẫn đến sự bùng nổ dữ liệu từ mạng xã hội, thiết bị IoT và nhiều nguồn khác. Điều này đặt ra thách thức cho các tổ chức trong việc lưu trữ, xử lý và tận dụng dữ liệu để đưa ra quyết định chiến lược. Trong bối cảnh đó, NoSQL xuất hiện như một giải pháp quản lý dữ liệu phi quan hệ, cho phép lưu trữ và truy vấn dữ liệu phi cấu trúc với khả năng mở rộng linh hoạt. Bên cạnh đó, Apache Spark là một nền tảng xử lý dữ liệu phân tán và song song, cung cấp hiệu suất cao cho các tác vụ phức tạp như xử lý dữ liệu thời gian thực, phân tích đồ thị, học máy và dữ liệu văn bản.

Bài báo cáo này tập trung vào việc nghiên cứu tính chất, nguồn gốc và ứng dụng của Big Data, đồng thời khám phá lý do tại sao xử lý dữ liệu lớn trở thành nhiệm vụ quan trọng đối với các tổ chức. Hơn thế nữa, kiến trúc và các thành phần chính của Apache Spark sẽ được phân tích để làm rõ lợi ích và khả năng ứng dụng của nó trong việc xử lý dữ liệu lớn. NoSQL cũng được đề cập như một công cụ hỗ trợ lưu trữ dữ liệu hiệu quả trong các hệ thống phân tán.

Mục tiêu của bài báo cáo này là giúp độc giả hiểu rõ hơn về Big Data, NoSQL và Apache Spark, đồng thời khám phá tiềm năng của chúng trong việc tạo ra giá trị và ứng dụng thực tiễn trong nhiều lĩnh vực như khoa học, kinh tế, y tế và quản lý. Qua đó, người

độc sẽ nhận thức được vai trò quan trọng của công nghệ xử lý dữ liệu lớn trong việc thúc đẩy đổi mới và ra quyết định dựa trên dữ liệu.

2 Big Data

2.1 Định nghĩa

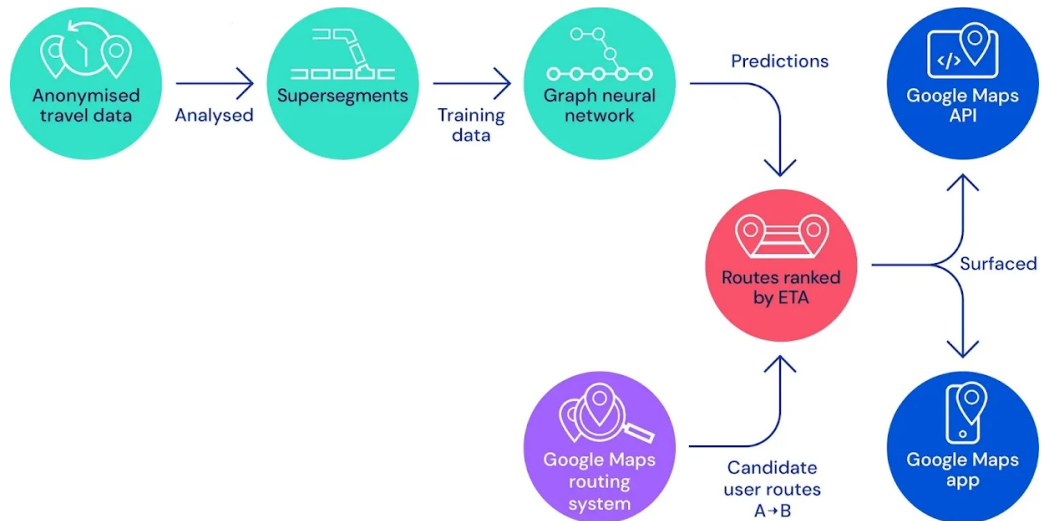
Thuật ngữ Big Data ra đời gắn liền với sự phát triển bùng nổ của khối lượng dữ liệu toàn cầu, cùng với nhu cầu xử lý và làm việc trên các tập dữ liệu khổng lồ. Trong cuộc cách mạng công nghiệp lần thứ tư, khái niệm Big Data được đề cập bên cạnh AI, IoT như một lĩnh vực mũi nhọn trong cuộc cách mạng chuyển đổi số mang tính đột phá.

Dữ liệu (data) là các tập hợp thông tin và sự kiện khác nhau, được thu thập, lưu trữ, xử lý và chia sẻ nhằm thu được các thông tin hữu ích. Dữ liệu có thể tồn tại dưới nhiều dạng khác nhau như chữ, số viết trên văn bản, hoặc các byte trong bộ nhớ máy tính, hoặc các sự kiện ghi lại trong trong trí nhớ con người. Data mang nhiều ý nghĩa và giá trị sử dụng tùy thuộc vào cách nó được khai thác. Dữ liệu được phân tích bằng các phương pháp đa dạng, từ đó được sử dụng cho việc nghiên cứu, đánh giá các hoạt động cụ thể trong nhiều lĩnh vực khác nhau như tài chính, giáo dục, y tế, v.v.

Big Data không chỉ là một xu hướng công nghệ mà còn là chìa khóa mở ra kỷ nguyên của sự hiểu biết sâu sắc và quyết định chính xác. Big data là các tập dữ liệu rất lớn và phức tạp, có cấu trúc hoặc phi cấu trúc và bán cấu trúc, rất khó để quản lý, lưu trữ và phân tích bằng các công cụ xử lý dữ liệu truyền thống. Điều quan trọng của Big Data là khả năng phân tích và tìm hiểu thông tin từ những tập dữ liệu này, là một công cụ quan trọng trong các lĩnh vực của đời sống, xã hội.

Ví dụ: Hình 41, Google Maps đã tận dụng sức mạnh của phân tích dữ liệu lớn (Big Data) để cung cấp cho người dùng tuyến đường tối ưu nhất bằng cách xử lý dữ liệu theo thời gian thực từ nhiều nguồn khác nhau. Ứng dụng này thu thập thông tin từ vị trí và tốc độ di chuyển của hàng triệu thiết bị di động, giúp xác định mật độ giao thông trên từng đoạn đường. Bên cạnh đó, dữ liệu từ các cảm biến giao thông và camera đường phố cho phép Google Maps nắm bắt tình hình ùn tắc cũng như các sự kiện bất thường. Lịch sử lưu lượng giao thông theo khung giờ, ngày trong tuần hoặc trong các sự kiện đặc biệt cũng được tận dụng để dự đoán xu hướng tắc đường. Google Maps có thể dự đoán và đề xuất tuyến đường ít tắc nhất, giúp người dùng tiết kiệm thời gian và hỗ trợ quản lý giao

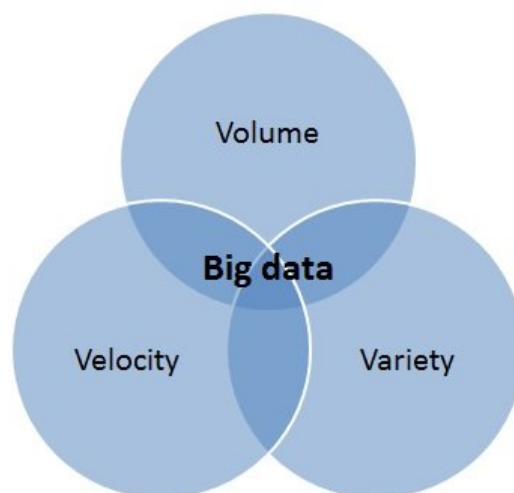
thông đô thị hiệu quả hơn.



Hình 1: Ứng dụng Google Maps sử dụng Big Data để tối ưu hóa giao thông.

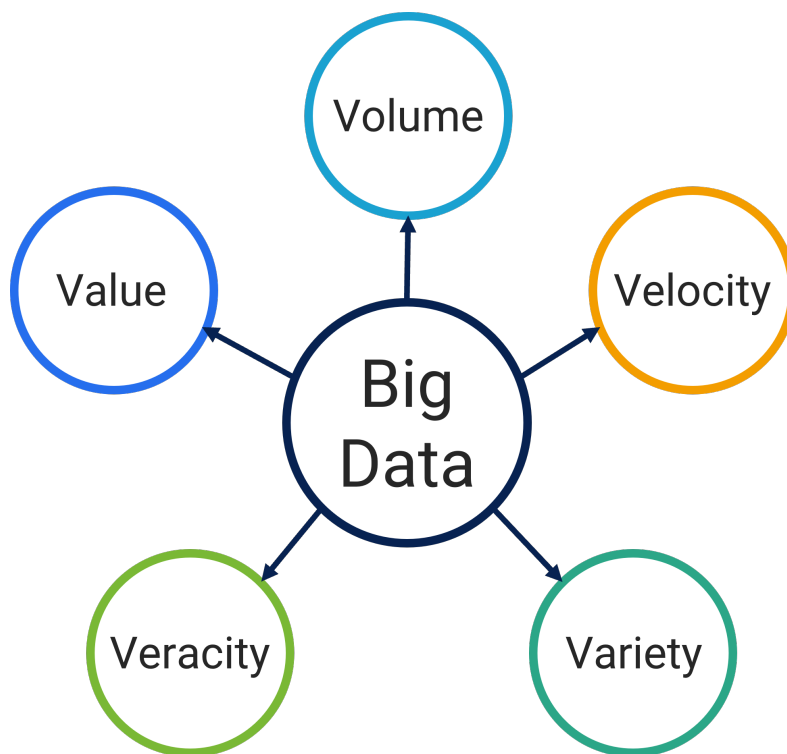
2.2 Đặc trưng

Big Data không chỉ đơn giản là một khối lượng dữ liệu lớn, mà còn là một hệ sinh thái phức tạp có nhiều đặc trưng quan trọng. Big Data bắt đầu với 3 đặc trưng cơ bản, được gọi là mô hình 3V là Volume (Khối lượng dữ liệu), Velocity (Tốc độ dữ liệu), Variety (Đa dạng).



Hình 2: Mô hình 3V.

Tuy nhiên, khi Big Data phát triển mạnh mẽ, mô hình đã mở rộng ra, bổ sung thêm hai đặc trưng nữa tạo nên mô hình 5V là Volume (Khối lượng dữ liệu), Velocity (Tốc độ nhanh), Variety (Đa dạng), Veracity (Tính chính xác) và Value (Giá trị). Đây cũng là các đặc trưng thường được mô tả cho Big Data.



Hình 3: Mô hình 5V.

- **Volume (Khối lượng dữ liệu):** Khối lượng của Big Data là rất lớn, thậm chí là cực lớn thường được đo bằng terabyte (TB), petabyte (PB) hoặc thậm chí zettabyte (ZB)... Kích thước của dữ liệu đóng vai trò rất quan trọng trong việc xác định giá trị, tiềm năng của dữ liệu. Ngoài ra, việc một dữ liệu cụ thể có thực sự được coi là Big Data hay không, phụ thuộc vào khối lượng của dữ liệu.
- **Velocity (Tốc độ xử lý):** Mô tả tốc độ dữ liệu được tạo ra và tốc độ di chuyển của dữ liệu, tốc độ xử lý của dữ liệu. Đây là một khía cạnh quan trọng đối với các tổ chức (tài chính, ngân hàng. . .) cần dữ liệu của họ chạy nhanh, sát với thời gian thực, để dữ liệu có sẵn vào đúng thời điểm để đưa ra các quyết định tốt nhất có thể.
- **Variety (Đa dạng):** Trong Big Data đề cập đến các loại dữ liệu khác nhau được thu thập từ nhiều nguồn khác nhau. Dữ liệu được chia thành ba loại chính: Dữ liệu phi cấu trúc: Không có định dạng cụ thể, khó tích hợp vào cơ sở dữ liệu truyền thống

(hình ảnh, âm thanh, video. . .). Dữ liệu bán cấu trúc: Chưa được tổ chức hoàn chỉnh nhưng có thông tin bổ sung (metadata) giúp xử lý dễ hơn. Dữ liệu cấu trúc: Được tổ chức trong kho lưu trữ định dạng, thuận lợi cho xử lý và phân tích (dữ liệu trong bảng, cơ sở dữ liệu quan hệ).

- **Veracity (Tính chính xác):** Đề cập đến chất lượng, độ chính xác, tính toàn vẹn và độ tin cậy của dữ liệu. Dữ liệu thu thập được có thể bị thiếu sót, không chính xác hoặc có chứa dữ liệu dị thường thường được gọi là nhiễu. Nói chung, Veracity thể hiện mức độ tin cậy mà một tổ chức có đối với dữ liệu đã thu thập.
- **Value (Giá trị):** Đề cập đến lợi ích và giá trị thực tiễn mà Big Data mang lại, liên quan trực tiếp đến cách các tổ chức tận dụng dữ liệu đã thu thập. Khả năng khai thác giá trị từ Big Data là điều cần thiết và là mục tiêu quan trọng, đem lại hiệu quả cao trong công việc.

Không dừng lại ở đó, hiện nay trong một số lĩnh vực còn mở rộng mô hình 7V, bổ sung thêm hai yếu tố mới là Variability (Tính biến động) và Visualization (Trực quan hóa). Đề cập đến sự thay đổi của dữ liệu theo thời gian và trực quan hóa dữ liệu trở nên dễ dàng hơn.

- **Variability (Biến động):** Đề cập đến sự thay đổi và không nhất quán của dữ liệu theo thời gian, khiến việc quản lý và phân tích trở nên phức tạp hơn. Lưu lượng dữ liệu có thể dao động đáng kể, đòi hỏi các hệ thống phải linh hoạt và có khả năng mở rộng để xử lý hiệu quả.
- **Visualization (Trực quan hóa):** Là khả năng chuyển đổi dữ liệu phức tạp thành các biểu đồ, đồ thị và báo cáo dễ hiểu. Trực quan hóa dữ liệu không chỉ giúp truyền tải thông tin một cách rõ ràng mà còn hỗ trợ người ra quyết định nắm bắt nhanh chóng các xu hướng và kết quả phân tích.

2.3 Các phương pháp xử lý dữ liệu phổ biến

Quy trình xử lý Big Data: Bao gồm các bước từ thu thập, lưu trữ, tiền xử lý, phân tích đến trực quan hóa và ra quyết định. Dữ liệu được thu thập từ nhiều nguồn khác nhau như cảm biến, mạng xã hội, thiết bị IoT và hệ thống doanh nghiệp, sau đó được lưu trữ trong các hệ thống lớn có khả năng mở rộng như Hadoop HDFS hoặc Amazon S3. Trước

khi phân tích, dữ liệu thô cần được làm sạch và chuẩn hóa để loại bỏ lỗi và sự không nhất quán. Các công cụ như Apache Spark hoặc TensorFlow được sử dụng để phân tích dữ liệu, trích xuất thông tin hữu ích nhằm hỗ trợ dự đoán và ra quyết định. Kết quả phân tích được trình bày qua biểu đồ và báo cáo bằng các công cụ như Tableau hoặc Power BI, giúp các tổ chức đưa ra quyết định chiến lược. Ngoài ra, dữ liệu quan trọng được lưu trữ dài hạn và bảo mật để đảm bảo an toàn và tuân thủ quy định.

2.3.1 Batch processing

- Khái niệm:

- + Là phương pháp xử lý dữ liệu lớn theo từng nhóm (batch), có dung lượng lớn, lặp đi lặp lại một cách tự động, đòi hỏi rất ít tương tác của con người. Phương pháp này giúp tối ưu hóa hiệu suất, tiết kiệm thời gian và tài nguyên cho máy tính bằng cách cho phép xử lý nhiều dữ liệu cùng lúc. Phù hợp với các tác vụ không yêu cầu thời gian thực.
- + Các công cụ hỗ trợ Batch processing: Hadoop, Apache Spark, Hive, MapReduce. . .

- Ưu điểm:

- + *Thời gian xử lý linh hoạt*: Các nhóm dữ liệu được xử lý ngoài giờ cao điểm, thường vào cuối ngày hoặc ban đêm, giúp giảm tải cho hệ thống trong giờ cao điểm và tối ưu việc sử dụng tài nguyên.
- + *Tự động hóa cao*: Quá trình xử lý diễn ra hoàn toàn tự động, không phụ thuộc vào yếu tố con người. Điều này giúp giảm thiểu công sức và chi phí quản lý hệ thống, đồng thời cho phép người dùng tập trung vào các công việc khác mà không cần giám sát hoặc can thiệp vào quá trình xử lý.
- + *Tính nhất quán và đáng tin cậy*: Batch processing xử lý dữ liệu theo tuần tự nhất định, giảm thiểu lỗi và đảm bảo kết quả nhất quán trong mỗi lần xử lý. Điều này đặc biệt quan trọng với các tác vụ yêu cầu độ chính xác cao.
- + *Khả năng xử lý dữ liệu lớn*: Batch processing phù hợp với các tác vụ cần xử lý một lượng lớn dữ liệu, tận dụng sức mạnh tính toán để hoàn tất trong thời gian ngắn hơn so với xử lý từng phần riêng lẻ.

Một số mô hình lập trình và framework xử lý dữ liệu được sử dụng để xử lý khối lượng dữ liệu lớn một cách hiệu quả như:

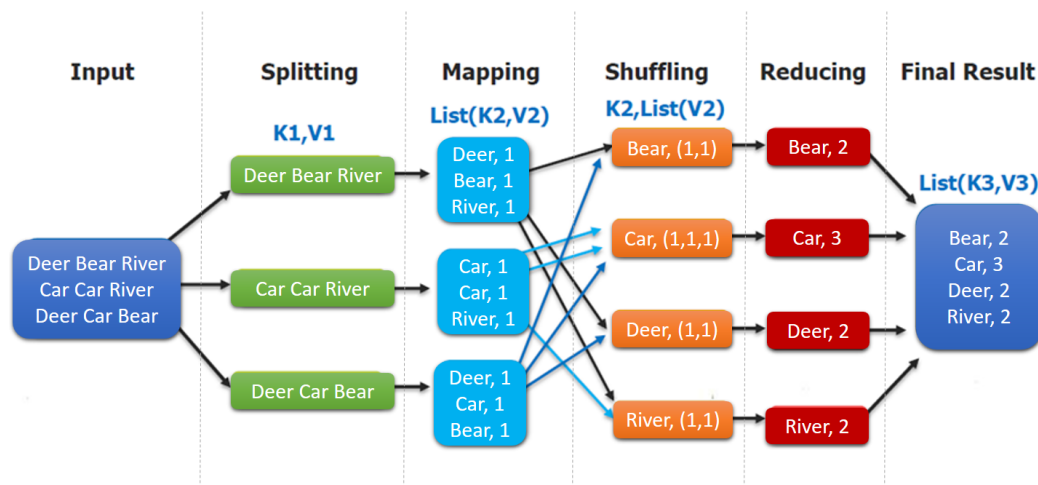
- *Apache Hadoop MapReduce*: Xử lý dữ liệu phân tán trên nhiều nút mạng. theo hai bước Map và Reduce, phù hợp với hệ thống phân tán lớn nhưng độ trễ cao.
- *Apache Spark*: Xử lý dữ liệu nhanh nhờ sử dụng bộ nhớ trong (in-memory), cho phép xử lý cả batch lẫn stream.

- **Ứng dụng:**

- + *Phân tích dữ liệu quy mô lớn*: Với khả năng xử lý dữ liệu theo lô, Batch Processing rất phù hợp cho việc phân tích các tập dữ liệu lớn. Các doanh nghiệp thường sử dụng để phân tích lưu lượng truy cập web, thông tin khách hàng và kết quả bán hàng nhằm hỗ trợ ra quyết định.
- + *Xử lý giao dịch tài chính*: Các ngân hàng và hệ thống tài chính sử dụng Batch Processing để xử lý hàng loạt giao dịch trong ngày, như chuyển tiền, rút tiền và thanh toán hóa đơn. Việc tổng hợp và xử lý dữ liệu vào cuối ngày giúp đảm bảo độ chính xác và hiệu suất cao.
- + *Sao lưu và phục hồi dữ liệu*: Batch Processing được sử dụng trong việc sao lưu dữ liệu định kỳ vào cuối ngày hoặc vào những thời điểm ngoài giờ cao điểm. Điều này giúp bảo vệ dữ liệu quan trọng và đảm bảo tính sẵn sàng khi cần khôi phục. Nếu có sự cố hệ thống, dữ liệu này có thể được khôi phục đầy đủ để đảm bảo hoạt động không bị gián đoạn.

- **Giới thiệu về MapReduce:**

- + MapReduce là một kiến trúc xử lý dữ liệu phân tán được thiết kế để xử lý các tập dữ liệu lớn trên quy mô rộng. Ban đầu được phát triển bởi Google, kiến trúc này sau đó đã được Apache Hadoop áp dụng và phát triển, trở thành một trong những công nghệ cốt lõi trong xử lý dữ liệu lớn. MapReduce hoạt động bằng cách chia quy trình xử lý thành hai giai đoạn chính: Map và Reduce.
- + Trong giai đoạn Map, dữ liệu đầu vào được chia thành các bản ghi nhỏ hơn và xử lý đồng thời trên các nút trong cụm máy tính. Kết quả của giai đoạn này là các cặp khóa-giá trị trung gian. Ở giai đoạn Shuffle: các cặp khóa-giá trị trung gian có cùng giá trị sẽ được gom lại vào một key. Tiếp theo, trong giai đoạn Reduce, các cặp khóa-giá trị này được tổng hợp và xử lý để tạo ra kết quả cuối cùng. Điều này cho phép xử lý dữ liệu song song trên nhiều máy tính, giúp giảm thiểu thời gian và tăng hiệu suất cho các tác vụ xử lý dữ liệu lớn.



Hình 4: Mô hình MapReduce trong bài toán đếm số từ.

2.3.2 Stream processing (Real - time processing)

- Khái niệm:

- + Là phương pháp xử lý dữ liệu theo luồng các sự kiện trong thời gian thực, xử lý dữ liệu liên tục hoặc gần như ngay lập tức khi dữ liệu được tạo ra. Thay vì chờ tích lũy đủ dữ liệu để xử lý theo lô (batch processing), dữ liệu trong stream processing được xử lý ngay khi nó xuất hiện. Điều này cho phép hệ thống phản ứng kịp thời với các sự kiện và cung cấp thông tin liên tục, phù hợp cho các ứng dụng yêu cầu thời gian thực.
- + Các công cụ hỗ trợ Stream processing: Apache Kafka, Apache Flink, Storm, Google Cloud Dataflow...

- Đặc điểm:

- + *Xử lý liên tục với độ trễ thấp*: Dữ liệu được xử lý ngay khi xuất hiện, cho phép đưa ra kết quả gần như ngay lập tức.
- + *Khả năng mở rộng và phân tán*: Hỗ trợ xử lý trên hệ thống phân tán, có thể mở rộng để xử lý lượng dữ liệu lớn.
- + *Độ tin cậy và tuần tự*: Đảm bảo xử lý dữ liệu theo thứ tự và giữ tính nhất quán ngay cả khi xảy ra sự cố.

- Ứng dụng:

- + *Phân tích dữ liệu theo thời gian thực*: Stream Processing giúp các tổ chức phân tích dữ liệu ngay lập tức khi dữ liệu được tạo ra, cho phép họ có cái nhìn tức thì về hoạt động kinh doanh.
- + *Xử lý dữ liệu máy móc*: Stream Processing giúp xử lý dữ liệu máy móc trong thời gian thực, bao gồm dữ liệu từ cảm biến và log hệ thống. Điều này giúp tăng cường hiệu suất, khả năng dự báo lỗi, và quản lý hiệu quả hơn. *Gian lận trong thể thao*: Có thể gắn các con chip vào giày chạy, khi vận động viên vượt qua các cảm biến đặt dọc đường, chúng ta có thể sử dụng thông tin đó để theo dõi vị trí người chạy từ đó phát hiện ra gian lận hay sự cố người đó gặp phải.

2.3.3 Hyper processing

- Khái niệm:

- + Là mô hình xử lý dữ liệu tiên tiến, cho phép xử lý lượng dữ liệu khổng lồ với độ trễ cực thấp (near-zero latency) và hiệu suất cao. Không giống như Batch Processing (xử lý theo lô) hay Stream Processing (xử lý theo thời gian thực), Hyper Processing kết hợp cả tốc độ và tính nhất quán, đảm bảo dữ liệu được xử lý gần như ngay lập tức.
- + Các công cụ hỗ trợ: Aerospike, Apache Ignite, Hazelcast, Redis...

- Đặc điểm:

- + *Độ trễ cực thấp*: Dữ liệu được xử lý gần như ngay lập tức với độ trễ chỉ vài mili-giây.
- + *Xử lý đồng thời*: Hỗ trợ xử lý lượng lớn dữ liệu và hàng triệu sự kiện cùng lúc.
- + *Tính sẵn sàng cao*: Đảm bảo hệ thống hoạt động liên tục, không bị gián đoạn.

2.3.4 So sánh các phương thức

Yếu tố	Batch processing	Stream processing	Hyper processing
Đối tượng	File tĩnh hoặc dữ liệu đã lưu trữ	Luồng sự kiện liên tục	Luồng dữ liệu khối lượng lớn trong thời gian thực
Thời gian xử lý	Xử lý định kỳ theo lịch trình cố định	Xử lý gần như ngay lập tức khi dữ liệu xuất hiện	Xử lý ngay lập tức với độ trễ cực thấp
Đầu vào	Dữ liệu được lưu trữ trước khi xử lý	Dữ liệu đến liên tục và được xử lý ngay khi xuất hiện	Dữ liệu khối lượng lớn được xử lý song song ngay khi đến
Đầu ra	Kết quả được xuất ra sau khi xử lý toàn bộ dữ liệu	Kết quả được xuất ra liên tục theo thời gian thực	Kết quả được xuất ra ngay lập tức với hiệu suất cực cao
Ứng dụng	Thích hợp cho xử lý dữ liệu lớn, không yêu cầu thời gian thực	Thích hợp phân tích dữ liệu trực tuyến, phát hiện gian lận	Thích hợp cho ứng dụng đòi hỏi hiệu suất cực cao như giao dịch tài chính

Bảng 1: So sánh các phương thức xử lý dữ liệu

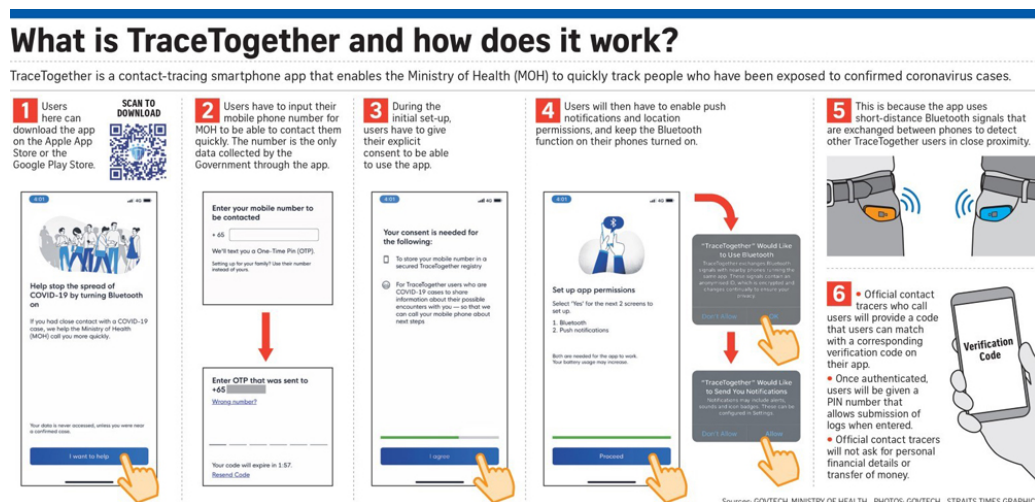
2.4 Ứng dụng của Big Data

Big Data giúp các doanh nghiệp và tổ chức đưa ra quyết định nhanh chóng, chính xác hơn, tối ưu hóa quy trình và nâng cao trải nghiệm người dùng. Big Data gần như ứng dụng được ở tất cả các lĩnh vực.

- Y tế

Big Data đã và đang tạo ra cuộc cách mạng trong lĩnh vực y tế, mang đến những bước đột phá trong việc chẩn đoán bệnh tật, chăm sóc sức khỏe bệnh nhân hay quản lý các bệnh viện. Thực tế cho thấy, việc ứng dụng Big Data trong y tế đem lại những hiệu quả đáng kể, từ việc giảm chi phí chữa bệnh, điều trị và phòng ngừa bệnh tật cho đến việc nâng cao chất lượng cuộc sống, nhất là khi việc áp dụng Hồ sơ sức khỏe điện tử (EHR) ngày càng phổ biến.

- + **Chẩn đoán và điều trị bệnh tật:** Big Data cho phép thu thập và phân tích lượng dữ liệu lớn từ các kết quả xét nghiệm, hình ảnh chụp X-quang, MRI, tiền sử bệnh án hay các chỉ số sức khỏe liên quan đến từ bệnh nhân. Qua đó, các thuật toán học máy có thể phân tích các mẫu dữ liệu nói trên để phát hiện các dấu hiệu của bệnh tật một cách chính xác hơn. Các bác sĩ có thể căn cứ kết quả phân tích kể trên để thiết kế phác đồ điều trị sao cho phù hợp và hiệu quả với bệnh nhân.
- + **Quản lý hồ sơ sức khỏe điện tử:** Big Data cho phép tích hợp các thông tin sức khỏe cơ bản như cân nặng, chiều cao, nhóm máu, cho đến tiền sử bệnh án, kết quả xét nghiệm hay những thông tin quan trọng khác vào trong hồ sơ sức khỏe điện tử cá nhân. Việc sử dụng hồ sơ đã mang lại những lợi ích đáng kể như dễ dàng quản lý thông tin bệnh nhân, tiết kiệm thời gian truy cập, nâng cao sự tương tác giữa bệnh nhân và bác sĩ. Có thể thấy, hồ sơ sức khỏe điện tử mang lại những hiệu quả nhất định hơn so với hồ sơ giấy thông thường.
- + **Hệ thống chăm sóc sức khỏe thông minh:** Thông qua những thiết bị theo dõi sức khỏe điện tử tại nhà, chẳng hạn như đồng hồ sức khỏe đeo tay, AI có thể xử lý dữ liệu thu thập được từ người dùng trong thời gian thực như nhịp tim, tâm lý, nhiệt độ cơ thể, rồi đưa ra những báo cáo về sức khỏe, có thể là hàng giờ và người dùng có thể dựa trên những báo cáo trên để theo dõi sức khỏe một cách chủ động, và có thể lên kế hoạch thăm khám nếu cần thiết.



Hình 5: Ứng dụng Trace Together của Singapore sử dụng BigData truy vết người mắc bệnh trong đại dịch Covid-19.

- Giáo dục

Big Data đã tạo ra một cuộc cách mạng trong lĩnh vực giáo dục, mở ra nhiều cơ hội để nâng cao chất lượng giảng dạy, cá nhân hóa quá trình học tập và tối ưu hóa quản lý nhà trường. Việc ứng dụng Big Data giúp phát hiện sớm các vấn đề học tập, hỗ trợ học sinh vượt qua khó khăn và định hướng phát triển phù hợp với từng cá nhân, từ đó cải thiện kết quả học tập và giảm tỷ lệ bỏ học.



Hình 6: Phân bố học lực học sinh được tổng hợp tự động bởi hệ thống VnEdu, áp dụng Big Data và AI.

- + **Cá nhân hóa quá trình học tập:** Big Data cho phép thu thập và phân tích hàng loạt dữ liệu liên quan đến quá trình học tập của học sinh, chẳng hạn như điểm số, hành vi trong lớp, thời gian làm bài tập và cả tương tác trực tuyến trên các nền tảng học tập. Các hệ thống sử dụng dữ liệu của học sinh để điều chỉnh nội dung bài giảng phù hợp với khả năng và nhu cầu riêng của từng người.
- + **Đánh giá năng lực học tập:** Thông qua việc phân tích dữ liệu từ điểm số, thời gian tham gia lớp học, và hành vi học tập trực tuyến, hệ thống Big Data có thể dự đoán sớm những học sinh có nguy cơ bỏ học hoặc gặp khó khăn trong quá trình học tập. Big Data giúp các trường học xác định các vấn đề chung trong quá trình giảng dạy, từ đó có chiến lược cải thiện chương trình học, đào tạo giáo viên và phân bổ nguồn lực hợp lý hơn.
- + **Tối ưu hóa phân bổ nguồn lực:** Quản lý nhà trường hiệu quả không chỉ liên quan đến học tập mà còn bao gồm việc phân bổ tài nguyên như phòng học, giáo viên, trang thiết bị và ngân sách. Điển hình như việc Quản lý lịch học linh hoạt để tránh tình trạng lớp học quá tải hoặc thiếu phòng học. Dự đoán nhu cầu nhân

sự giáo viên dựa trên số lượng học sinh đăng ký. Tối ưu hóa chi phí hoạt động, như tiết kiệm năng lượng, mua sắm trang thiết bị phù hợp với nhu cầu thực tế.

- Tài chính

- + *Quản lý rủi ro*: Hệ thống Big Data thu thập và lưu trữ dữ liệu ở thời gian thực giúp các ngân hàng, công ty tài chính phát hiện sớm các biến động bất thường trên thị trường, từ đó đưa ra những biện pháp xử lý kịp thời.
- + *Phát hiện gian lận tài chính*: Big Data cho phép giám sát hàng triệu giao dịch tài chính trong thời gian thực. Các ngân hàng có thể truy cập lịch sử giao dịch trong thời gian thực và phát hiện các giao dịch bất thường có liên quan đến gian lận từ đó ngăn chặn việc thực hiện các giao dịch có yếu tố gian lận. Ví dụ, để giải quyết gian lận hiệu quả, tập đoàn Alibaba đã xây dựng hệ thống giám sát và quản lý rủi ro dựa trên xử lý của Big Data theo thời gian thực. Hệ thống sẽ xác định các giao dịch xấu và nắm bắt các tín hiệu gian lận bằng cách phân tích một lượng lớn dữ liệu về hành vi của người dùng trong thời gian thực bằng cách sử dụng các thuật toán học máy.
- + *Phân tích hành vi của khách hàng*: Các ngân hàng đang đẩy mạnh việc sử dụng Big Data nhằm phân tích và đánh giá về thói quen chi tiêu, sở thích, các hành vi trực tuyến và phản ứng của khách hàng, từ đó có thể nắm bắt được nhu cầu chi tiêu phổ biến của khách hàng, cũng như dự đoán hành vi của khách hàng trong tương lai. Nhờ đó mà các ngân hàng có thể đưa ra các dịch vụ phù hợp với xu hướng của khách hàng.

2.5 Tương lai và thách thức trong việc phát triển Big Data

2.5.1 Tương lai của Big Data

- **Phát triển trí tuệ nhân tạo (AI) và Machine Learning (ML)**: Big Data là nền tảng quan trọng cho AI và ML. Các thuật toán học máy ngày càng phụ thuộc vào lượng dữ liệu lớn để cải thiện độ chính xác và khả năng dự đoán.
- **Internet of Things (IoT)**: Với sự bùng nổ của thiết bị IoT, lượng dữ liệu được tạo ra sẽ tiếp tục tăng theo cấp số nhân. Big Data sẽ đóng vai trò quan trọng trong việc xử lý và phân tích dữ liệu theo thời gian thực từ các thiết bị này.

- **Cá nhân hóa trải nghiệm người dùng:** Big Data giúp các doanh nghiệp hiểu rõ hơn về hành vi và sở thích của khách hàng, từ đó cung cấp trải nghiệm cá nhân hóa tốt hơn.
- **Tối ưu hóa kinh doanh:** Các công ty sẽ ngày càng sử dụng Big Data để tối ưu hóa hoạt động, dự đoán xu hướng thị trường và đưa ra quyết định kinh doanh hiệu quả hơn.

2.5.2 Thách thức trong việc phát triển Big Data

- **Quản lý và lưu trữ dữ liệu:** Lượng dữ liệu ngày càng lớn đặt ra thách thức trong việc lưu trữ, quản lý và tối ưu chi phí.
- **Bảo mật và quyền riêng tư:** Dữ liệu lớn đi kèm với rủi ro về bảo mật thông tin cá nhân. Các tổ chức phải đối mặt với việc tuân thủ các quy định về quyền riêng tư và bảo vệ dữ liệu.
- **Xử lý và phân tích dữ liệu:** Dữ liệu lớn có thể phức tạp, không đồng nhất và khó xử lý. Cần các công cụ và kỹ thuật tiên tiến để trích xuất thông tin hữu ích.
- **Nguồn nhân lực:** Thiếu nhân lực có kỹ năng chuyên môn cao trong lĩnh vực Big Data là một rào cản lớn đối với nhiều tổ chức.

2.6 Kết luận

Big Data là một trong những lĩnh vực mũi nhọn hiện nay và trong tương lai, mang lại lợi ích to lớn cho doanh nghiệp và cá nhân. Nó mở ra nhiều cơ hội trong việc tối ưu hóa kinh doanh, cá nhân hóa trải nghiệm và hỗ trợ ra quyết định thông minh. Tuy nhiên, bên cạnh những lợi ích, Big Data cũng đi kèm với các thách thức về thu thập, lưu trữ và xử lý khối lượng dữ liệu khổng lồ một cách hiệu quả và nhanh chóng.

Với sự ra đời và phát triển của các công cụ xử lý dữ liệu tiên tiến như NoSQL, Hadoop và Apache Spark, việc xử lý dữ liệu lớn, phức tạp đang trở nên nhanh chóng và hiệu quả hơn. Những công nghệ này giúp các tổ chức không chỉ lưu trữ mà còn phân tích và trích xuất thông tin hữu ích từ dữ liệu trong thời gian thực.

Trong tương lai, Big Data sẽ tiếp tục đóng vai trò quan trọng trong việc thúc đẩy sự đổi mới trong nhiều lĩnh vực. Tuy nhiên, để tận dụng tối đa tiềm năng của Big Data, các tổ chức cần chú trọng đến việc đầu tư cơ sở hạ tầng, bảo mật dữ liệu và đào tạo nguồn nhân lực chất lượng cao.

3 NoSQL

3.1 Kỷ nguyên BigData và các vấn đề của RDBMS

Hệ quản trị cơ sở dữ liệu quan hệ (Relational Database Management System – RDBMS) là một loại phần mềm quản lý dữ liệu, trong đó dữ liệu được tổ chức dưới dạng các bảng (tables) có cấu trúc rõ ràng, được liên kết với nhau thông qua các khóa (keys).

RDBMS được xây dựng dựa trên mô hình quan hệ do E. F. Codd đề xuất vào năm 1970, sử dụng các khái niệm toán học như tập hợp và đại số quan hệ để mô hình hóa dữ liệu và các thao tác dữ liệu.

Đặc điểm chính:

- + Dữ liệu được lưu trữ trong các bảng (gồm hàng và cột).
- + Hỗ trợ SQL (Structured Query Language) – ngôn ngữ truy vấn có cấu trúc.
- + Đảm bảo tính nhất quán và toàn vẹn dữ liệu.
- + Tuân thủ mô hình ACID để bảo toàn giao dịch.

RDBMS hiện vẫn là xương sống dữ liệu của hàng loạt hệ thống ứng dụng quan trọng trong doanh nghiệp như: ngân hàng, tài chính, quản lý nhân sự, thương mại điện tử,...

Những khó khăn và hạn chế của RDBMS

- Hạn chế trong xử lý dữ liệu phi cấu trúc
 - + RDBMS được thiết kế để lưu trữ dữ liệu có cấu trúc (structured data) dưới dạng bảng (table) với schema xác định rõ ràng.
 - + Trong khi đó, Big Data bao gồm nhiều loại dữ liệu: phi cấu trúc (video, ảnh, văn bản thô), bán cấu trúc (JSON, XML), và cấu trúc.
 - + Điều này khiến RDBMS không phù hợp để lưu trữ và xử lý log hệ thống, dữ liệu cảm biến IoT, dữ liệu mạng xã hội, v.v.
- Khó khăn khi mở rộng theo chiều ngang (Horizontal Scaling)

- + RDBMS thường mở rộng theo chiều dọc (Vertical Scaling) — tức là nâng cấp phần cứng máy chủ.
- + Việc này tốn kém, giới hạn về phần cứng, và không phù hợp với nhu cầu xử lý hàng petabyte dữ liệu phân tán trên nhiều khu vực địa lý.
- + Trong khi đó, các hệ thống Big Data cần khả năng phân tán và chia tải (distributed & sharding) mạnh mẽ.
- Giới hạn về hiệu suất trong xử lý khối lượng dữ liệu lớn
 - + Khi số lượng bản ghi tăng lên hàng tỷ, hiệu suất của RDBMS suy giảm rõ rệt trong các thao tác như JOIN, GROUP BY, hoặc truy vấn phức tạp.
 - + Các ứng dụng phân tích dữ liệu lớn (Big Data Analytics) như MapReduce, Spark, Flink không thể tích hợp hiệu quả với kiến trúc RDBMS truyền thống.
- Khó thích nghi với thay đổi dữ liệu (schema evolution)
 - + RDBMS yêu cầu schema cố định → mỗi thay đổi về bảng, cột sẽ ảnh hưởng đến toàn bộ hệ thống, gây gián đoạn và mất thời gian bảo trì.
 - + Trong môi trường phát triển nhanh, linh hoạt như hiện nay, điều này giới hạn khả năng thích ứng.
- Không tối ưu cho tính sẵn sàng cao và phân tán
 - + RDBMS thường duy trì tính nhất quán chặt chẽ (ACID), khiến nó khó áp dụng mô hình phân tán đa trung tâm.
 - + Khi có sự cố về mạng hoặc node, việc đảm bảo tính toàn vẹn dữ liệu có thể khiến hệ thống bị ngưng trệ.
 - + Điều này trái ngược với yêu cầu của Big Data là sẵn sàng cao (High Availability) và chịu lỗi (Fault Tolerance).
- Không đáp ứng được các mô hình truy cập đặc thù
 - + Một số ứng dụng như mạng xã hội, hệ thống đề xuất, tìm kiếm văn bản full-text, cần loại dữ liệu phi truyền thống như cặp khóa-giá trị (key-value), tài liệu (document), đồ thị (graph).
 - + RDBMS không hỗ trợ tốt cho những mô hình dữ liệu phi quan hệ này.

NoSQL ra đời là để giải quyết các hạn chế của RDBMS trong bối cảnh bùng nổ dữ liệu của kỷ nguyên Big Data

3.2 Sơ lược về NoSQL

Thuật ngữ NoSQL được giới thiệu lần đầu vào năm 1998 sử dụng làm tên gọi chung cho các lightweight open source relational database (cơ sở dữ liệu quan hệ nguồn mở nhỏ) nhưng không dùng SQL cho truy vấn. Vào năm 2009, nhân viên của Rackspace giới thiệu lại thuật ngữ NoSQL trong hội thảo về cơ sở dữ liệu nguồn mở phân tán.

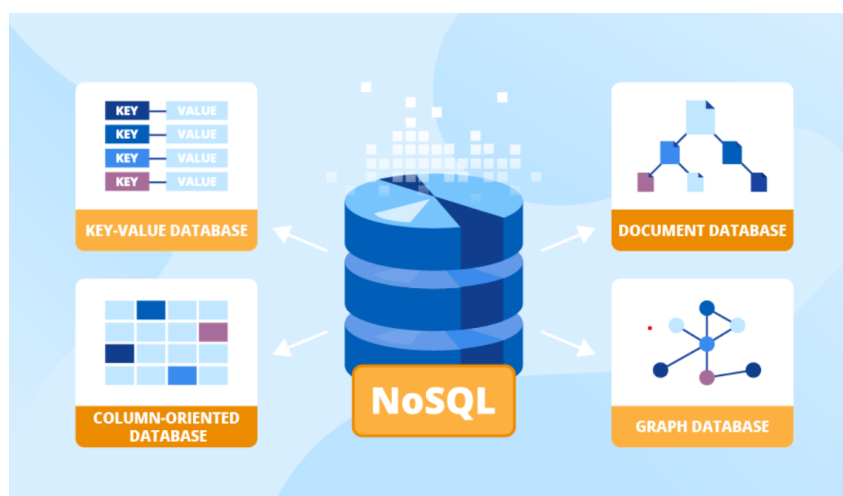
Thuật ngữ NoSQL đánh dấu bước phát triển của thế hệ database mới: distributed (phân tán) + non-relational (không ràng buộc). Đây là 2 đặc tính quan trọng nhất.

NoSQL (Not Only SQL) là *hệ thống quản lý cơ sở dữ liệu phi quan hệ*, được thiết kế để lưu trữ và truy vấn dữ liệu có cấu trúc linh hoạt, phân tán, và có khả năng mở rộng cao. Phát triển NoSQL để đáp ứng nhu cầu lưu trữ và xử lý dữ liệu phi cấu trúc của Big Data.

Một số ứng dụng như: Xử lý dữ liệu lớn (BigData), lưu trữ thông tin người dùng,...

3.3 Các loại NoSQL

Có 4 loại chung (phổ biến nhất) của cơ sở dữ liệu NoSQL.



Hình 7: Các loại hình NoSQL Database.

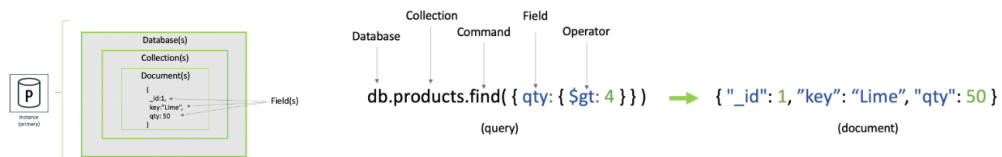
3.3.1 Key-value store



Hình 8: Ví dụ về Key-value store.

- Là kiểu lưu trữ dữ liệu NoSQL đơn giản nhất sử dụng từ một API, thường được sử dụng cho các hệ thống yêu cầu truy xuất nhanh và có khả năng mở rộng cao. Dữ liệu được lưu trữ dưới dạng các cặp key-value.
- Các đặc điểm của cơ sở dữ liệu key-value: Nhất quán (Consistency), Hỗ trợ giao dịch (Transaction), Tính năng truy vấn (Query Feature), Cấu trúc dữ liệu (Data Structure) Mở rộng (Scaling).
- Một vài cơ sở dữ liệu key-value phổ biến là Riak, Redis(thường dùng phía server), memcached, Berkeley DB, HamsterDB, Amazon DynamoDB(mã nguồn đóng), Project Voldemort và Couchbase.

3.3.2 Document store

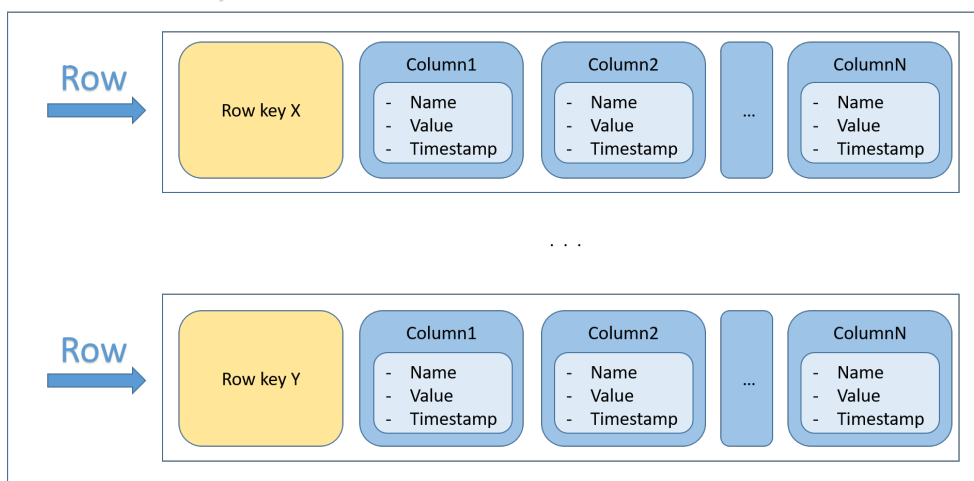


Hình 9: Ví dụ về Document store

- Document Database lưu trữ dữ liệu dưới dạng *tài liệu (document)*, thường ở định dạng *JSON, BSON, hoặc XML*. Mỗi tài liệu là một thực thể độc lập, giúp lưu trữ linh hoạt mà không cần tuân theo schema cố định như RDBMS.
- Mỗi tài liệu trong cơ sở dữ liệu Document là độc lập và không thể thực hiện các phép nối (join) mà trong ngữ cảnh này có nghĩa là không có cơ chế tự động để duy trì tính toàn vẹn quan hệ giữa các phần dữ liệu khi thực hiện các phép nối.
- Mặc dù không hỗ trợ dữ liệu quan hệ và bên cạnh các ưu điểm tương tự như cơ sở dữ liệu key-value, cơ sở dữ liệu Document có những ưu điểm bổ sung như khả năng truy vấn không chỉ bằng key mà còn truy vấn thuộc tính (attribute) trong tài liệu. Dữ liệu trong mỗi tài liệu có thể ở định dạng khác nhau và có thể được truy vấn một cách linh hoạt.
- Một số cơ sở dữ liệu Document phổ biến là MongoDB, CouchDB và Lotus Notes.

3.3.3 Column-Family Store

Column family



Hình 10: Ví dụ về Column-Family store

- Column-Family Databases *lưu trữ dữ liệu theo cột* thay vì hàng, giúp tối ưu hiệu suất truy vấn và tổng hợp dữ liệu. *Mở rộng theo chiều ngang*, phù hợp với hệ thống phân tán và xử lý dữ liệu lớn (Big Data).
- Cơ sở dữ liệu Column Family được biểu diễn như một hệ thống lưu trữ Key-value đa chiều, tương tự như một bản đồ hoặc một mảng kết hợp (associative array). Dữ

liệu trong cơ sở dữ liệu này không chỉ bền vững, tức là giá trị vẫn tồn tại sau khi được tạo ra hoặc truy cập (access), mà còn được phân tán trên nhiều nút (node) tính toán và lưu trữ, giúp tối ưu hóa hiệu suất và khả năng mở rộng. Các key được sắp xếp để hỗ trợ quá trình truy vấn, và dữ liệu có thể bị khuyết như trong cơ sở dữ liệu quan hệ.

- Có hai loại Column Family:

+ **Standard Column Family:** gồm một cặp key-value, trong đó key được ánh xạ tới một giá trị (value) là một tập hợp các cột.

+ **Super Column Family:** bao gồm một cặp key-value, trong đó key được ánh xạ tới một giá trị (value) là các column family.

- Một số cơ sở dữ liệu Column Family phổ biến bao gồm Google's Bigtable, Cassandra, và HBase. Cơ sở dữ liệu dạng cột *rất mạnh* khi xử lý dữ liệu lớn và truy vấn phân tích nhưng *không phù hợp* cho các hệ thống yêu cầu *ACID* và truy vấn phức tạp.

3.3.4 Graph Database



Hình 11: Ví dụ về Graph Database

- Cơ sở dữ liệu Graph là hệ thống lưu trữ dữ liệu theo hướng đồ thị. Nó lưu dữ liệu dưới dạng nút (node) đại diện cho đối tượng, cạnh (edge) thể hiện mối quan hệ giữa các nút, thuộc tính (property) lưu trữ thông tin của các nút và cạnh.

- Do Graph Database xây dựng dựa trên lý thuyết đồ thị nên việc truy vấn dữ liệu trong loại cơ sở dữ liệu này thường áp dụng các thuật toán duyệt đồ thị phổ biến như tìm kiếm theo chiều sâu (DFS) hay tìm kiếm theo chiều rộng (BFS).
- Một số cơ sở dữ liệu Graph phổ biến gồm Neo4j, OrientDB và Allegrograph.
- Cơ sở dữ liệu đồ thị (Graph Database) truy vấn quan hệ phức tạp nhanh, linh hoạt, phù hợp cho mạng xã hội, gợi ý sản phẩm, phát hiện gian lận. Nhưng khó mở rộng quy mô lớn, không mạnh về ACID, yêu cầu kỹ năng quản lý chuyên sâu.

3.4 So sánh NoSQL với SQL

3.4.1 Tổng quan về SQL và NoSQL

- **SQL**
 - + SQL là một ngôn ngữ truy vấn được thiết kế để tương tác với các hệ thống quản lý cơ sở dữ liệu quan hệ (RDBMS – Relational Database Management Systems), chẳng hạn như MySQL, PostgreSQL, Oracle, SQL Server,...
 - + Ngôn ngữ truy vấn SQL được sử dụng hiệu quả trong việc thêm mới, truy xuất, cập nhật hoặc xóa các bản ghi trong cơ sở dữ liệu quan hệ.
- **NoSQL**
 - + NoSQL là một cơ sở dữ liệu không quan hệ, không yêu cầu một lược đồ cố định, tránh các phép nối và dễ dàng mở rộng. Cơ sở dữ liệu NoSQL được sử dụng cho các kho dữ liệu phân tán với nhu cầu lưu trữ dữ liệu khổng lồ.
 - + Hiểu một cách đơn giản, NoSQL là ngược lại so với SQL. NoSQL được sử dụng với mục đích tương đối giống như SQL, nhưng là đối với cơ sở dữ liệu không quan hệ. Cơ sở dữ liệu không quan hệ có thể gồm Big Data (dữ liệu lớn) và các web application thời gian thực.

3.4.2 So sánh sự khác nhau giữa SQL và NoSQL

Tiêu chí	SQL	NoSQL
Loại hình	SQL databases là cơ sở dữ liệu dựa trên bảng và các bảng này có quan hệ với nhau	NoSQL databases có thể dựa trên nhiều hướng lưu trữ khác nhau: document-oriented, column-oriented, graph-based, ...
Ngôn ngữ truy vấn	Sử dụng SQL để truy vấn và quản lý dữ liệu. Các câu lệnh như SELECT, INSERT, UPDATE, DELETE được sử dụng để tương tác với dữ liệu.	Ngôn ngữ truy vấn đa dạng. Các cơ sở dữ liệu NoSQL sử dụng các ngôn ngữ truy vấn không phải là SQL.
Khả năng mở rộng	Cơ sở dữ liệu SQL có thể được thu nhỏ theo chiều dọc, được mở rộng bằng cách tăng lưu lượng phần cứng.	Được tùy biến theo chiều ngang, mở rộng bằng cách tăng số lượng máy chủ cơ sở dữ liệu.
Mục đích sử dụng	Được thiết kế dành cho các ứng dụng xử lý giao dịch trực tuyến, trong giao dịch có độ ổn định cao và thích hợp để xử lý phân tích trực tuyến.	Được thiết kế phục vụ cho việc phân tích dữ liệu có cấu trúc chưa hoàn chỉnh.
Lựa chọn sử dụng	– Dữ liệu có mối quan hệ phức tạp. – Yêu cầu về tính ACID và tính toàn vẹn dữ liệu cao. – Lược đồ cố định, được định rõ trước.	– Dữ liệu bán cấu trúc và phi cấu trúc. – Khả năng mở rộng nhanh chóng. – Hiệu suất cao cho đọc và ghi. – Lược đồ động, linh hoạt và có thể thay đổi.

Mã nguồn	Kết hợp của các mã nguồn mở như Postgres & MySQL và thương mại như Oracle Database.	Open-source.
Hiệu suất	Hiệu suất thường tốt cho các truy vấn liên quan đến mối quan hệ giữa các bảng.	Hiệu suất cao trong việc truy cập và cập nhật toàn bộ document hoặc các trường cụ thể trong document.
ACID-BASE	ACID (Atomicity, Consistency, Isolation, Durability) là một chuẩn cho RDBMS.	BASE (Basically Available, Soft State, Eventually Consistent) là một mô hình của nhiều hệ thống NoSQL.
Ưu điểm	– Hỗ trợ ACID (Atomicity, Consistency, Isolation, Durability), đảm bảo tính toàn vẹn và nhất quán của dữ liệu. – Truy vấn dữ liệu dễ dàng và linh hoạt với ngôn ngữ truy vấn SQL chuẩn. – Dữ liệu có cấu trúc, giúp giảm thiểu việc lặp lại thông tin và tối ưu hóa hiệu suất truy vấn.	– Hỗ trợ quy mô lớn và dễ dàng mở rộng hệ thống. – Được thiết kế để xử lý dữ liệu bán và phi cấu trúc và linh hoạt trong việc thay đổi cấu trúc dữ liệu. – Hiệu suất cao khi truy vấn dữ liệu lớn và phức tạp.
Nhược điểm	– Hạn chế trong việc mở rộng quy mô lớn và xử lý dữ liệu phi cấu trúc. – Khó khăn trong việc thay đổi cấu trúc dữ liệu khi ứng dụng phát triển.	– Không hỗ trợ đầy đủ các tính năng ACID, có thể dẫn đến sự không nhất quán và toàn vẹn trong một số tình huống. – Cú pháp truy vấn và tương tác với dữ liệu phức tạp hơn so với SQL.

Bảng 2: So sánh sự khác nhau giữa SQL và NoSQL

3.5 Kết luận

NoSQL là một hệ quản trị cơ sở dữ liệu linh hoạt, được thiết kế để xử lý dữ liệu phi cấu trúc và bán cấu trúc với khả năng mở rộng cao. Với nhiều mô hình dữ liệu khác nhau như Key-Value, Document, Column-Family và Graph, NoSQL đáp ứng tốt nhu cầu của các ứng dụng hiện đại, đặc biệt là trong môi trường dữ liệu lớn và thời gian thực. NoSQL không thay thế hoàn toàn SQL, mà bổ sung cho SQL khi cần xử lý dữ liệu theo cách linh hoạt hơn. Việc lựa chọn NoSQL hay SQL phụ thuộc vào đặc điểm dữ liệu và yêu cầu của hệ thống. Trong nhiều trường hợp, việc kết hợp cả hai công nghệ có thể mang lại hiệu quả tối ưu, giúp tận dụng điểm mạnh của từng loại cơ sở dữ liệu.

4 Apache Hadoop

4.1 Giới thiệu về Apache Hadoop

Hadoop là một framework mã nguồn mở được sử dụng phổ biến để xử lý và lưu trữ khối dữ liệu lớn trên các cụm máy tính phân tán, được thiết kế để xử lý và lưu trữ dữ liệu lớn một cách hiệu quả, được phát triển để giải quyết các thách thức trong lĩnh vực Big Data mà các công nghệ cũ không thể đáp ứng.



Hình 12: Hadoop.

Hadoop có nguồn gốc từ thời kỳ đầu của World Wide Web. Khi Web phát triển lên hàng triệu rồi hàng tỷ trang, nhiệm vụ tìm kiếm và trả về kết quả tìm kiếm trở thành một trong những thách thức nổi bật nhất. Các công ty khởi nghiệp như Google, Yahoo và AltaVista bắt đầu xây dựng các khuôn khổ để tự động hóa kết quả tìm kiếm. Hadoop được tạo ra bởi Doug Cutting và Mike Cafarella vào năm 2005 dựa trên công nghệ Google File System và MapReduce của Google.

Năm 2008, Hadoop được phát hành dưới dạng mã nguồn mở thông qua Apache Software Foundation, cho phép các nhà phát triển và công ty khác sử dụng, sửa đổi và phân phối nó. Kể từ đó, Hadoop đã nhanh chóng phát triển và trở thành một tiêu chuẩn trong lĩnh vực lưu trữ và xử lý dữ liệu lớn.

Hadoop sử dụng mô hình phân tán để lưu trữ và xử lý dữ liệu trên các máy tính thông thường. Thay vì lưu trữ tất cả dữ liệu trên một máy chủ duy nhất, Hadoop phân chia dữ liệu thành các khối và lưu chúng trên nhiều máy tính. Điều này giúp tăng khả năng mở rộng, độ tin cậy và hiệu năng. Hadoop phân tán dữ liệu trên nhiều nút máy tính và xử lý

nó song song trên các nút. Điều này giúp giảm thời gian xử lý và tăng hiệu suất.

Ngày nay, Hadoop vẫn quan trọng trong việc xử lý dữ liệu lớn, đặc biệt là cho các tác vụ phân tích theo lô (batch processing) và lưu trữ dữ liệu. Tuy nhiên, phần yếu của Hadoop là tốc độ xử lý chậm vì phải dựa vào đĩa cứng và mô hình lập trình MapReduce phức tạp. Chính vì những hạn chế này mà **Apache Spark** ra đời, cung cấp giải pháp xử lý dữ liệu nhanh hơn.

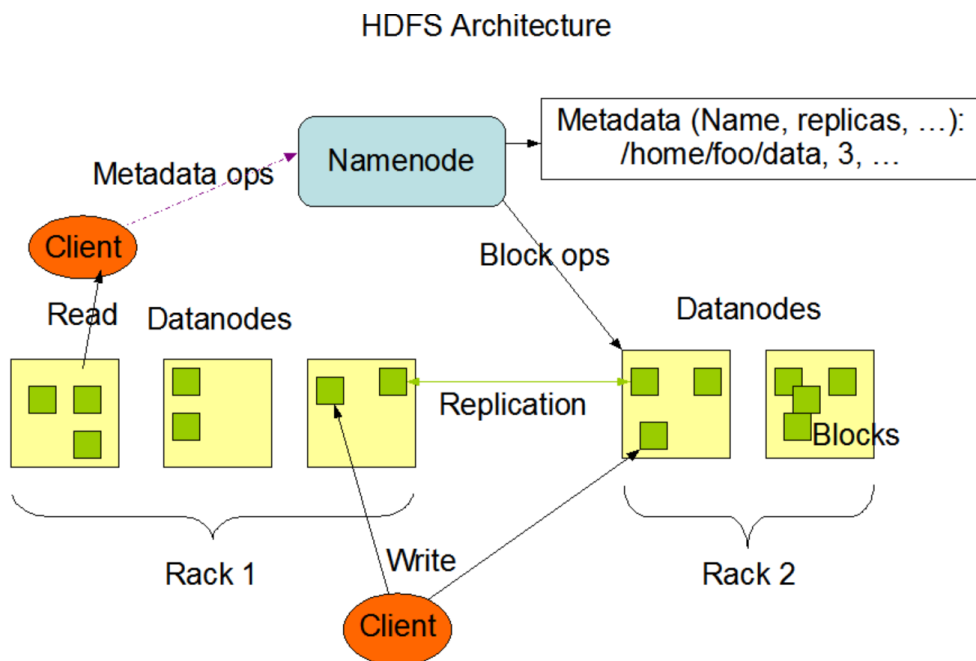
Tìm hiểu phần Hadoop Core (gồm HDFS và MapReduce), chúng ta hiểu được nguyên lý hoạt động cơ bản và những điểm yếu, từ đó lý giải tại sao **Apache Spark** được phát triển để khắc phục những vấn đề đó. Ngoài ra, trong hệ sinh thái của Hadoop còn có một số công cụ khác như YARN, Hive, HBase...

4.2 Kiến trúc và thành phần chính của Apache Hadoop

4.2.1 HDFS (Hadoop Distributed File System - Storage)

Trong thời đại dữ liệu lớn, các hệ thống lưu trữ truyền thống không còn đáp ứng được nhu cầu lưu trữ và xử lý dữ liệu khổng lồ. Vì vậy, HDFS ra đời như một giải pháp lưu trữ phân tán, đáng tin cậy và khả mở, giúp xử lý dữ liệu hiệu quả trên nhiều máy tính trong cùng một cụm.

- **HDFS (Hadoop Distributed File System)** là hệ thống quản lý tệp phân tán của Hadoop, dùng để lưu trữ dữ liệu lớn trên nhiều máy trong cụm (cluster). Nó được thiết kế để xử lý dữ liệu có dung lượng khổng lồ một cách hiệu quả, an toàn và bền bỉ. Ngoài ra, cung cấp tính năng tối ưu hoá việc sử dụng băng thông giữa các node. HDFS có thể được sử dụng để chạy trên một cluster lớn với hàng chục ngàn node.
- Là hệ thống tập tin ảo. Khi chúng ta di chuyển 1 tập tin trên HDFS, nó tự động chia thành nhiều mảnh nhỏ. Các đoạn nhỏ của tập tin sẽ được nhân rộng và lưu trữ trên nhiều máy chủ khác để tăng sức chịu lỗi và tính sẵn sàng cao.
- HDFS sử dụng kiến trúc master/slave, trong đó master gồm một NameNode để quản lý hệ thống file metadata và một hay nhiều slave DataNodes để lưu trữ dữ liệu thực tại.

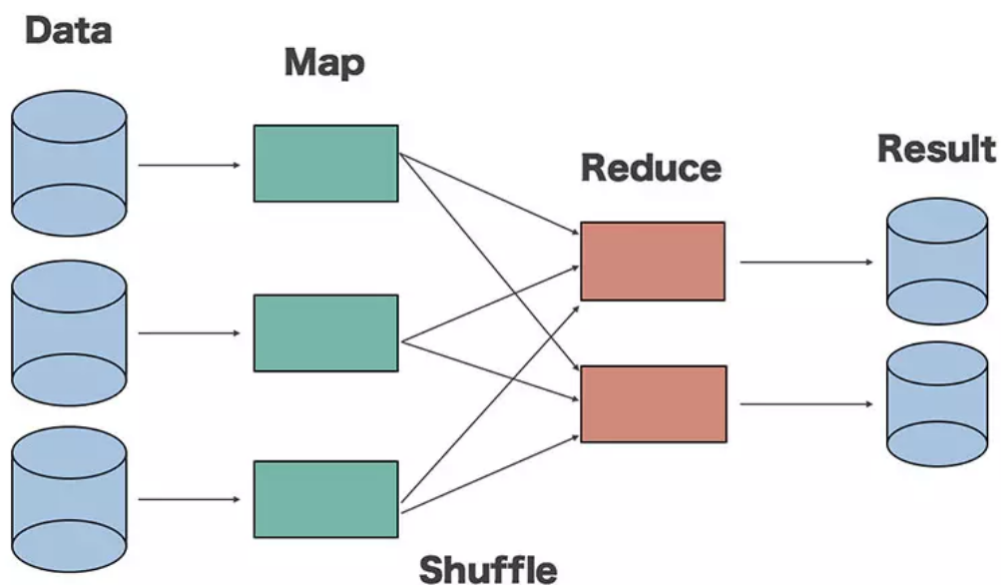


Hình 13: HDFS Architecture.

- + **NameNode**: Chạy trên máy chủ Master, chịu trách nhiệm quản lý namespace (không gian tên của hệ thống tệp) và điều phối các truy cập tệp từ phía client. Đây là trung tâm điều phối, nhưng không lưu dữ liệu thực tế.
- + **DataNode**: Chạy trên các nút Slave, có tác vụ lưu trữ dữ liệu thực tế dưới dạng các khối. Một tập tin với định dạng HDFS được chia thành nhiều khối (128MB hoặc 256MB) và những khối này được lưu trữ trong một tập các DataNodes. Các DataNode điều hành các tác vụ đọc và ghi dữ liệu lên hệ thống file thông qua các chỉ thị từ NameNode.
- **NameNode** và **DataNode** là hai thành phần chính trong HDFS. **NameNode** quản lý metadata như vị trí các block và thông tin về các tệp, đồng thời điều phối các yêu cầu từ client. Nó không lưu trữ dữ liệu thực tế mà chỉ quản lý namespace và phân quyền truy cập. **DataNode**, ngược lại, chịu trách nhiệm lưu trữ dữ liệu thực tế dưới dạng các block và xử lý các yêu cầu đọc/ghi từ client. Khi client yêu cầu truy cập dữ liệu, **NameNode** cung cấp thông tin về các DataNode chứa các block dữ liệu, và client sẽ kết nối trực tiếp với các DataNode để thực hiện thao tác đọc hoặc ghi.

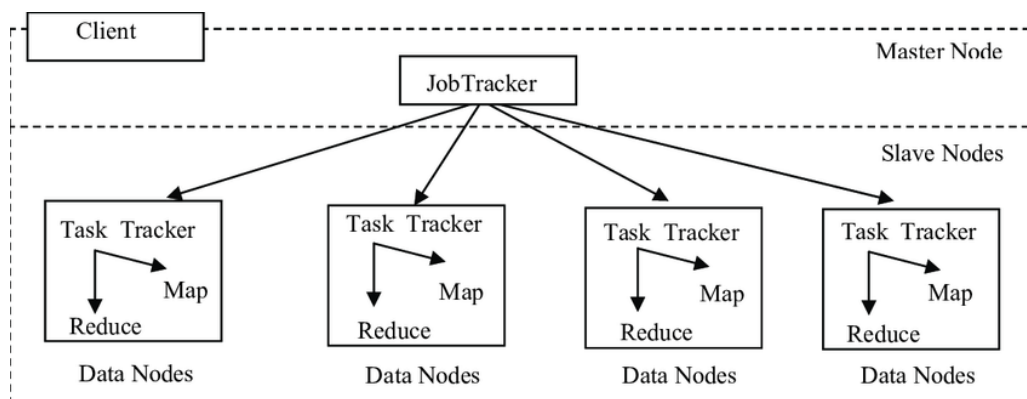
4.2.2 Map Reduce - Processing

- MapReduce được phát triển bởi Google và sau đó được Apache Software Foundation phát triển và phát hành dưới dạng một phần của hệ sinh thái Hadoop.
- MapReduce là hệ thống dựa trên YARN dùng để xử lý song song các tập dữ liệu lớn. Là cách chia một vấn đề dữ liệu lớn hơn thành các đoạn nhỏ hơn và phân tán nó trên nhiều máy chủ. Mỗi máy chủ có 1 tập tài nguyên riêng và máy chủ xử lý dữ liệu trên cục bộ. Khi máy chủ xử lý xong dữ liệu, chúng sẽ gửi trở về máy chủ chính.
- MapReduce bao gồm hai giai đoạn chính là pha Map và pha Reduce:
 - + *Map*: Thực hiện đầu tiên, có chức năng tải, phân tích dữ liệu đầu vào và được chuyển đổi thành tập dữ liệu theo cặp key/value.
 - + *Reduce*: Nhận kết quả đầu ra từ tác vụ Map, kết hợp dữ liệu lại với nhau thành tập dữ liệu nhỏ hơn.



Hình 14: MapReduce.

- MapReduce gồm một single master (máy chủ) JobTracker và các slave (máy trạm) TaskTracker trên mỗi cluster-node.

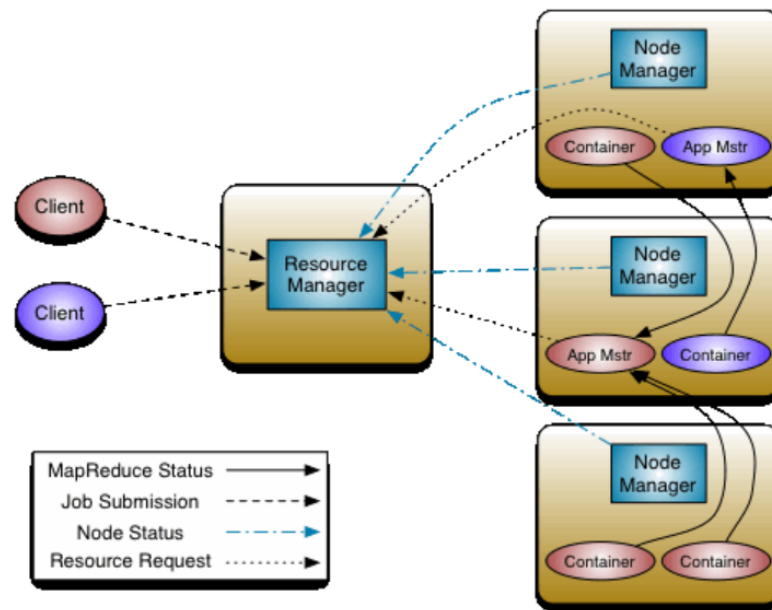


Hình 15: Jobtracker And Tasktracker.

- + *JobTracker*: Chạy trên một máy chủ (Master Node) và chịu trách nhiệm quản lý toàn bộ quá trình MapReduce. Nhận yêu cầu từ client và chia nhỏ công việc thành các Map tasks và Reduce tasks, sau đó phân phối chúng cho các TaskTracker.
- + *TaskTracker*: Chạy trên các máy trạm (Slave Nodes) và thực hiện các Map tasks và Reduce tasks mà JobTracker giao cho.

4.2.3 YARN (Yet Another Resource Negotiator) - Resource Management

- Hadoop YARN hoạt động như một framework hỗ trợ quản lý thư viện tài nguyên trong các cluster và thực hiện chạy các tiến trình phân tích, được giới thiệu trong Hadoop 2.x để thay thế JobTracker trong phiên bản cũ của Hadoop.
- YARN gồm ba thành phần chính:
 - + *ResourceManager (RM)*: Là master trong YARN quyết định các yêu cầu tài nguyên của ứng dụng và phân bổ tài nguyên cho các ApplicationMaster.
 - + *NodeManager (NM)*: Chạy trên các DataNode và quản lý tài nguyên của từng máy trạm (node) trong cụm Hadoop. Giám sát và báo cáo về tình trạng tài nguyên của các node cho ResourceManager (Ví dụ: dung lượng CPU, bộ nhớ,...).
 - + *ApplicationMaster (AM)*: Tạo ra các container (đơn vị tài nguyên), yêu cầu tài nguyên từ ResourceManager và phân phối công việc cho các Task trên NodeManager.

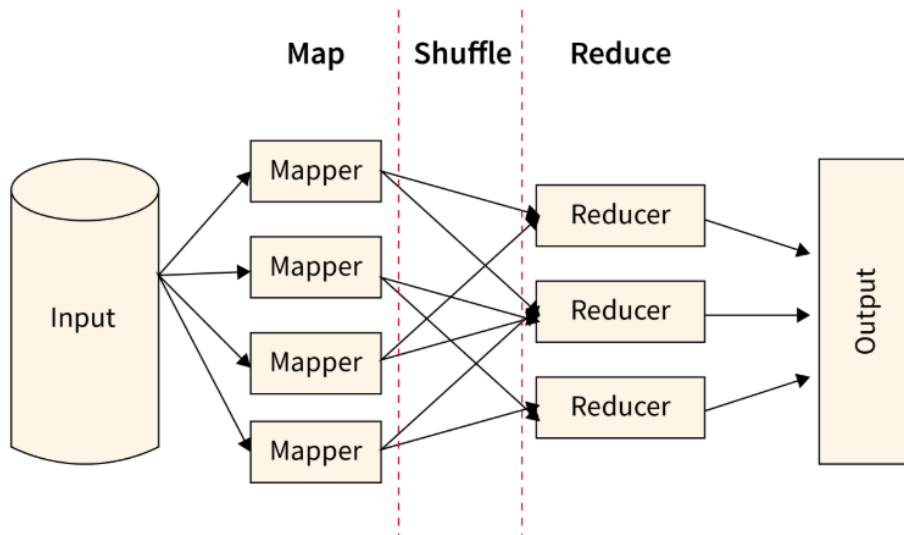


Hình 16: YARN.

4.3 Ứng dụng

- Hadoop được ứng dụng rộng rãi trong nhiều lĩnh vực để xử lý và phân tích dữ liệu lớn. Một số ứng dụng thực tế bao gồm:
 - + *Phân tích dữ liệu lớn*: Các công ty như Facebook, LinkedIn sử dụng Hadoop để phân tích hành vi người dùng và cải thiện trải nghiệm khách hàng.
 - + *Dự báo tài chính*: Các ngân hàng sử dụng Hadoop để phân tích dữ liệu giao dịch, dự báo xu hướng tài chính và phát hiện gian lận.
 - + *Khám phá gen và y tế*: Hadoop hỗ trợ phân tích dữ liệu gene và kết quả nghiên cứu y học để tìm ra phương pháp điều trị mới.

4.4 Hạn chế của Apache Hadoop



Hình 17: Xử lý chậm.

- **Xử lý chậm (Xử lý theo lô):** Hadoop sử dụng MapReduce, hoạt động theo chế độ xử lý theo lô, điều này khá chậm đối với các thuật toán lặp và xử lý dữ liệu theo thời gian thực. (Trong mỗi giai đoạn của MapReduce, dữ liệu phải được lưu trữ và đọc lại từ đĩa giữa các pha Map và Reduce. Điều này dẫn đến độ trễ cao vì mỗi lần cần phải thực hiện thao tác đọc/ghi vào đĩa, điều này làm giảm hiệu suất, đặc biệt là khi cần phải thực hiện nhiều vòng lặp. Đây là một hạn chế lớn đối với các trường hợp như học máy, phân tích thời gian thực và xử lý luồng dữ liệu, cần xử lý nhanh hơn.

Chính vì những hạn chế này của Hadoop đã dẫn đến sự ra đời và phát triển của các công nghệ mới như **Apache Spark**.

5 Apache Spark

5.1 Đặt vấn đề

Trong giai đoạn đầu của xử lý dữ liệu lớn, Apache Hadoop là một trong những nền tảng được sử dụng rộng rãi và đóng vai trò quan trọng nhờ khả năng xử lý song song trên quy mô lớn. Dựa trên mô hình lập trình MapReduce, Hadoop cho phép hệ thống dễ dàng mở rộng, chịu lỗi tốt và tiết kiệm chi phí. Tuy nhiên, mô hình này cũng bộc lộ một số hạn chế, đặc biệt trong bối cảnh các thuật toán học máy (machine learning) ngày càng phát triển. Cụ thể, MapReduce hoạt động theo luồng dữ liệu không tuần hoàn (acyclic data flow), nghĩa là dữ liệu đầu vào và đầu ra của mỗi tác vụ đều được lưu trữ trên bộ nhớ ổn định như ổ đĩa cứng. Khi cần tái sử dụng dữ liệu qua các vòng lặp – điều phổ biến trong các thuật toán học máy – hệ thống phải liên tục đọc và ghi dữ liệu từ đĩa, gây tốn thời gian và làm giảm hiệu suất xử lý.

Dù vậy, Hadoop vẫn được đánh giá cao nhờ khả năng mở rộng linh hoạt, chịu lỗi tốt và chi phí thấp – những yếu tố quan trọng trong xử lý dữ liệu quy mô lớn. Tuy nhiên, nhược điểm lớn nhất của Hadoop chính là mọi thao tác đều phải thực hiện trên đĩa cứng, khiến hiệu suất xử lý bị giới hạn nghiêm trọng. Để khắc phục vấn đề này, Apache Spark đã ra đời. Spark được thiết kế để hỗ trợ tính toán trong bộ nhớ (in-memory computing), giúp tăng tốc độ xử lý lên tới 10 lần so với Hadoop khi chạy trên đĩa cứng, và thậm chí nhanh hơn 100 lần khi xử lý hoàn toàn trong bộ nhớ RAM.

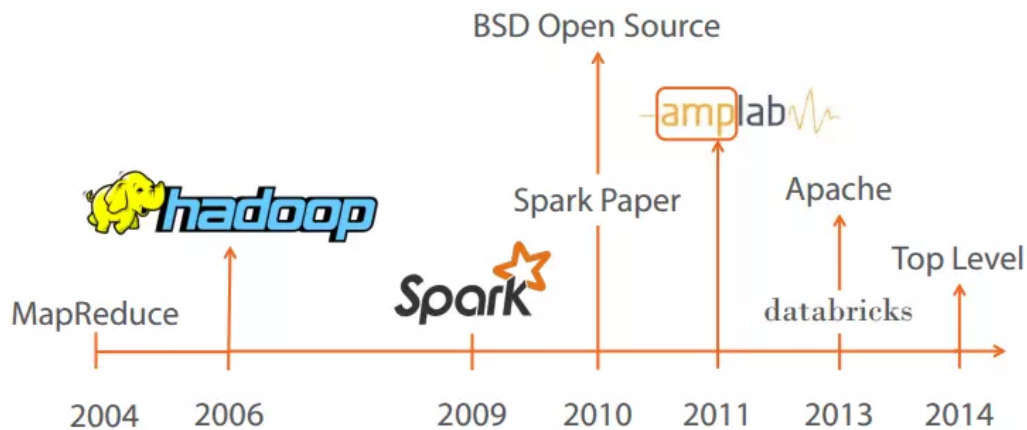
(Apache Spark ra đời nhằm cải thiện hiệu năng xử lý và hỗ trợ xử lý thời gian thực)

5.2 Apache Spark là gì

5.2.1 Sự ra đời

Apache Spark bắt đầu vào năm 2009 như một dự án nghiên cứu tại AMPLab của Đại học UC Berkeley, với sự tham gia của sinh viên, giảng viên và các nhà nghiên cứu. Mục tiêu ban đầu là cải thiện hiệu năng của Hadoop MapReduce bằng cách xây dựng một nền tảng có khả năng xử lý các tác vụ lặp lại nhanh chóng – như trong machine learning và

phân tích tương tác – đồng thời vẫn giữ được khả năng mở rộng và chịu lỗi giống như Hadoop. Spark được phát hành mã nguồn mở vào năm 2010 theo giấy phép BSD, và báo cáo nghiên cứu đầu tiên có tiêu đề “Spark: Cluster Computing with Working Sets” cũng được công bố cùng năm.



Hình 18: Lịch sử ra đời của Apache Spark.

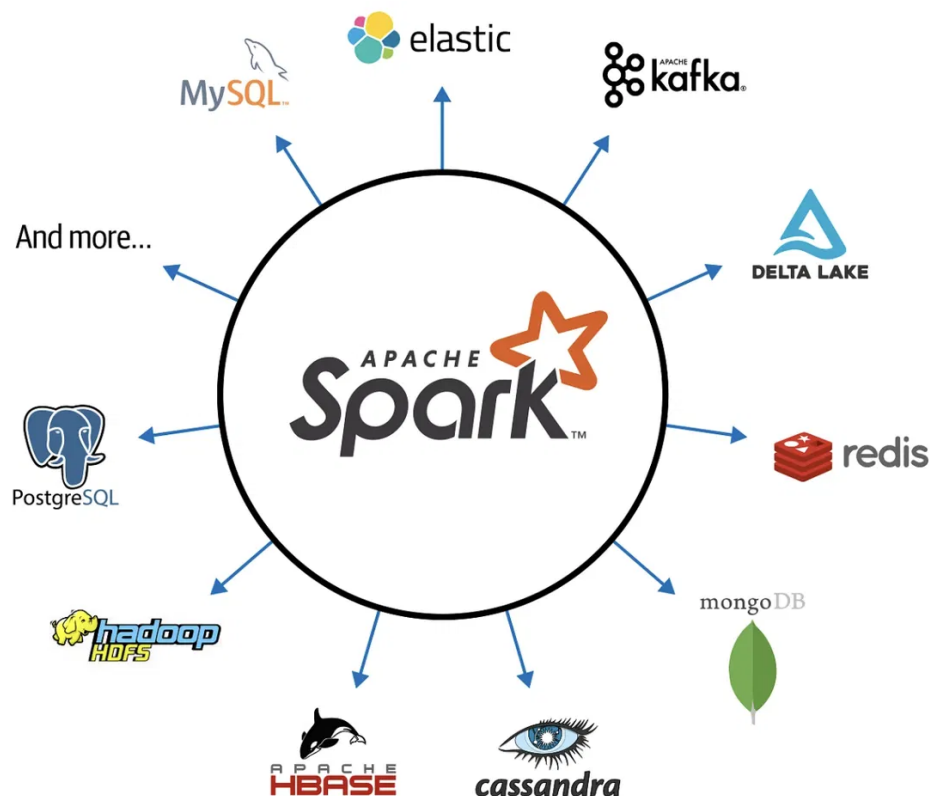
Đến tháng 6 năm 2013, Spark bắt đầu quá trình phát triển chính thức dưới sự bảo trợ của Quỹ Phần mềm Apache (ASF), dự án Spark có nhiều cá nhân đóng góp và phát triển, đến từ các công ty như Facebook, Yahoo!, Intel, Netflix, Databricks và nhiều đơn vị khác. Và được công nhận là Dự án Cấp cao nhất vào tháng 2 năm 2014. Kể từ đó, Spark đã trở thành một trong những nền tảng xử lý dữ liệu lớn, phổ biến trên thế giới. Nó có thể chạy độc lập, trên Apache Mesos hoặc tích hợp với hệ sinh thái Apache Hadoop.



Hình 19: Các doanh nghiệp sử dụng Apache Spark.

5.2.2 Apache Spark

- Apache Spark là một nền tảng xử lý dữ liệu lớn được phát triển bằng ngôn ngữ Scala, chạy trên nền tảng Java Virtual Machine (JVM) và sử dụng môi trường thực thi Java. Nhờ đó, Spark có tính tương thích cao, cho phép triển khai linh hoạt trên nhiều hệ điều hành như Windows và Linux.
- Là một Framework dùng để xử lý dữ liệu, Spark cung cấp các APIs cho các ngôn ngữ lập trình như Scala, Java, Python, R. Các API cấp cao của Spark giúp đơn giản hóa quá trình xử lý dữ liệu phân tán, giảm đáng kể lượng mã cần viết.
- Tốc độ xử lý vượt trội của Spark đến từ khả năng tính toán song song trên nhiều máy cùng lúc và việc xử lý dữ liệu được thực hiện trực tiếp trong bộ nhớ RAM (in-memory computing), giúp tăng hiệu năng gấp hàng chục lần so với các mô hình xử lý trên ổ đĩa như Hadoop MapReduce.



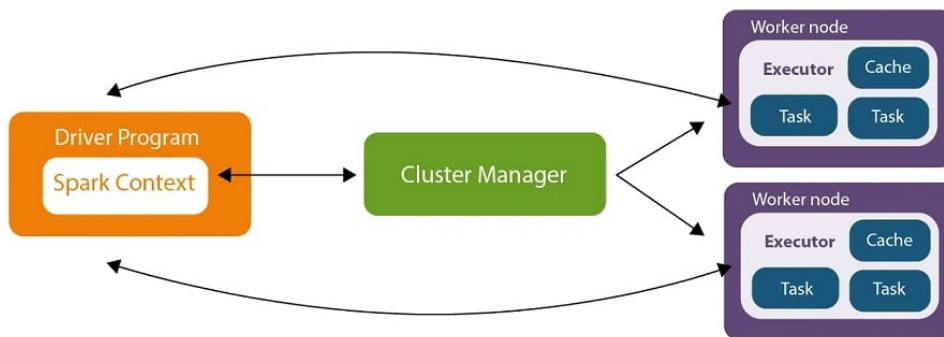
Hình 20: Apache Spark.

- Spark cũng hỗ trợ xử lý dữ liệu theo thời gian thực thông qua Spark Streaming – cho

phép hệ thống vừa tiếp nhận dữ liệu từ nhiều nguồn khác nhau, vừa xử lý ngay lập tức dữ liệu đầu vào. Ngoài ra, Spark không phụ thuộc vào một hệ thống tệp cụ thể, mà có thể hoạt động linh hoạt với nhiều hệ thống như HDFS, Cassandra, Amazon S3,... và hỗ trợ đa dạng các định dạng tệp như text, CSV, JSON,...

5.3 Kiến trúc của Apache Spark và cơ chế xử lý dữ liệu

5.3.1 Kiến trúc của Apache Spark



Hình 21: Kiến trúc của Apache Spark.

- Apache Spark là một framework xử lý dữ liệu phân tán mạnh mẽ và linh hoạt. Với kiến trúc phân tán, nó cho phép xử lý các tác vụ dữ liệu lớn và tính toán phân tán trên nhiều node. Kiến trúc của Apache Spark bao gồm các thành phần chính như Driver Program, Spark Context, Cluster Manager, Executors và Resilient Distributed Datasets (RDDs). Các thành phần này hoạt động cùng nhau để quản lý và thực thi quy trình xử lý dữ liệu trên một môi trường phân tán. Điều này giúp Apache Spark đạt được hiệu suất cao, khả năng mở rộng và độ tin cậy trong việc xử lý dữ liệu lớn và tính toán phân tán.

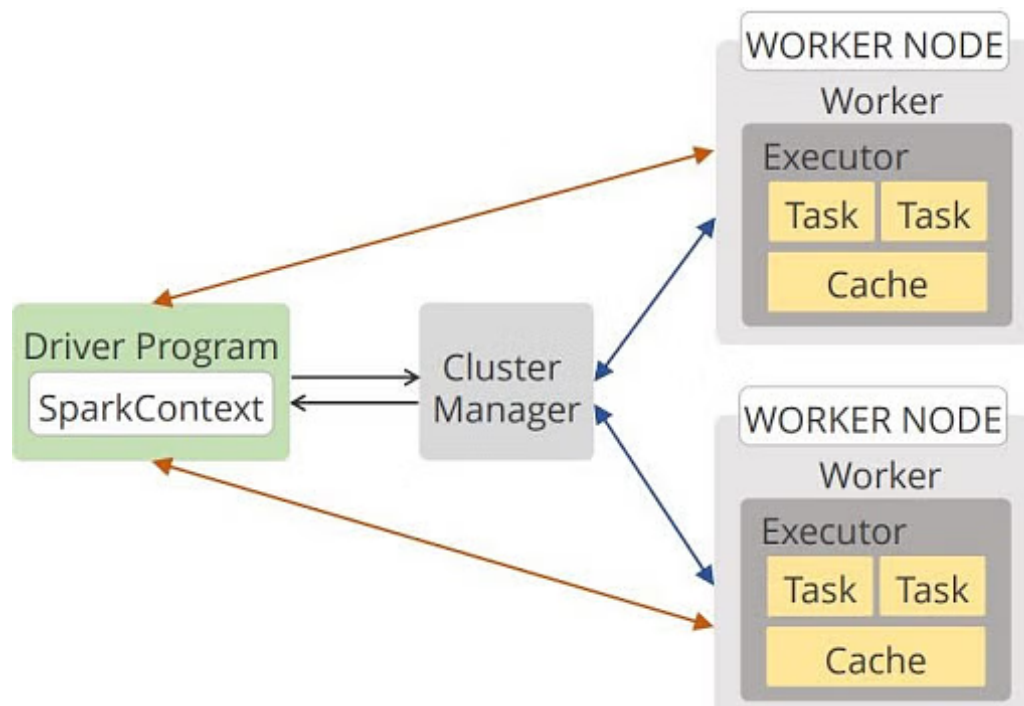
- + *Driver Program*: Đây là chương trình chính của ứng dụng Spark. Nó chạy trên một node trong cluster và quản lý quá trình xử lý trên toàn bộ cluster. Driver Program tạo và quản lý Spark Context.
- + *Spark Context*: Spark Context là một đối tượng quan trọng trong Spark. Nó đại diện cho kết nối với một cụm Spark và bao gồm các chức năng cơ bản như quản lý bộ nhớ, lập lịch tác vụ và khôi phục lỗi.

- + *Cluster Manager*: Cluster Manager giúp quản lý và phân phối tài nguyên trên các node trong cluster. Nó quản lý việc phân phối và giám sát quá trình xử lý trên các node để đảm bảo hiệu suất và độ tin cậy.
- + *Executors*: Executors là các tiến trình chạy trên các worker nodes trong cluster. Chúng được quản lý bởi driver program để thực hiện các tác vụ xử lý dữ liệu. Mỗi executor có thể chứa nhiều nhiệm vụ được giao để thực hiện.
- + *Resilient Distributed Datasets (RDDs)*: RDDs là cấu trúc dữ liệu cốt lõi trong Spark. Chúng là tập hợp dữ liệu phân tán và bất biến có thể được xử lý song song trên nhiều node. RDDs được tạo trong Spark Context, được phân phối trên các worker nodes và được lưu trữ trong bộ nhớ cache để tăng hiệu suất.
- Spark Driver, Spark Context, Cluster Manager, Executors và RDDs là các thành phần quan trọng trong kiến trúc của Apache Spark. Chúng cùng nhau tạo nên một hệ thống phân tán mạnh mẽ cho xử lý dữ liệu lớn và tính toán song song trên nhiều node.

5.3.2 Cơ chế xử lý dữ liệu

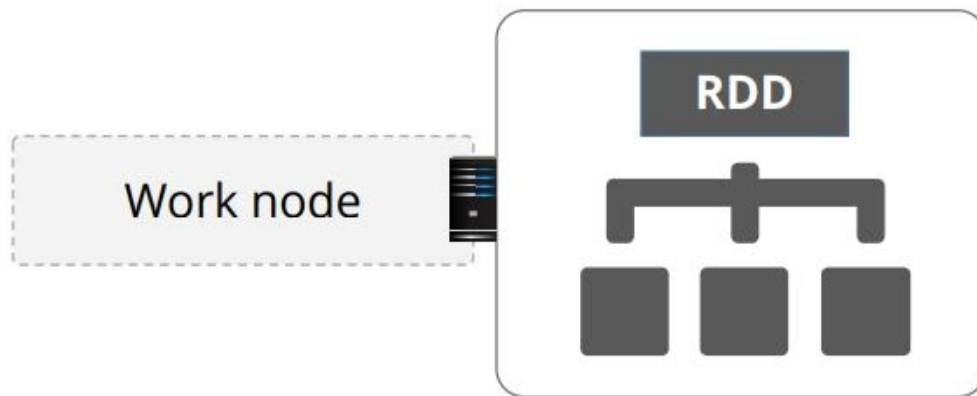
Với sự bùng nổ của dữ liệu lớn, nhu cầu về các hệ thống xử lý dữ liệu hiệu quả, nhanh chóng và có khả năng mở rộng quy mô ngày càng trở nên cấp thiết. Apache Spark là một trong những nền tảng mã nguồn mở nổi bật, hỗ trợ xử lý dữ liệu song song và phân tán mạnh mẽ thông qua mô hình lập trình dựa trên bộ nhớ trong (in-memory computing) và hệ sinh thái phong phú.

- Khi một ứng dụng Spark được khởi chạy, hệ thống sẽ tạo một tiến trình điều khiển gọi là Driver Program để quản lý toàn bộ quá trình thực thi. Driver này sẽ kết nối với Master Node của cụm Spark để phân chia công việc. Master Node chịu trách nhiệm phân phối tài nguyên và khởi tạo các Executor trên Worker Nodes. Các Executor là những tiến trình độc lập thực hiện các Task – đơn vị xử lý nhỏ nhất trong Spark.



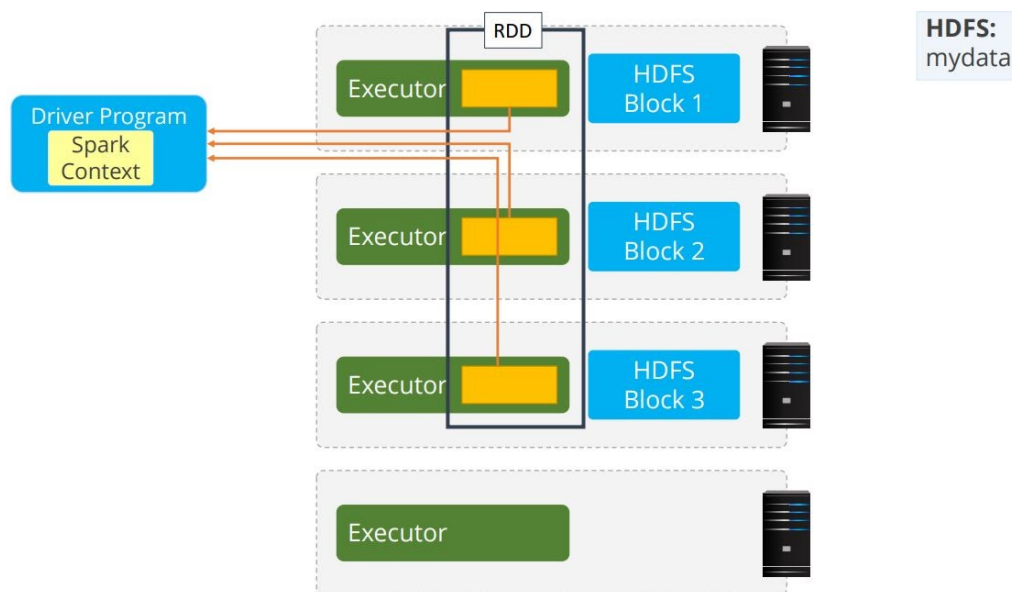
Hình 22: Cơ chế xử lý dữ liệu.

- Dữ liệu trong Spark được biểu diễn dưới dạng RDD (Resilient Distributed Dataset). RDD là một tập dữ liệu bất biến, có thể phân tán trên nhiều node trong cụm. Spark tự động chia nhỏ RDD thành nhiều partition, và các partition này có thể được xử lý song song trên nhiều Executor.
- Parallelism là một yếu tố cốt lõi trong cơ chế xử lý dữ liệu của Apache Spark, cho phép chia nhỏ dữ liệu thành các phân vùng để xử lý song song trên nhiều node trong cụm. Spark hỗ trợ cả Task Parallelism (chia nhỏ thao tác) và Data Parallelism (chia nhỏ dữ liệu), giúp tối ưu hiệu suất xử lý. Việc sử dụng RDDs và khả năng phân vùng động giúp Spark tận dụng tối đa tài nguyên, tăng tốc độ xử lý và đảm bảo độ tin cậy cao trong môi trường phân tán.



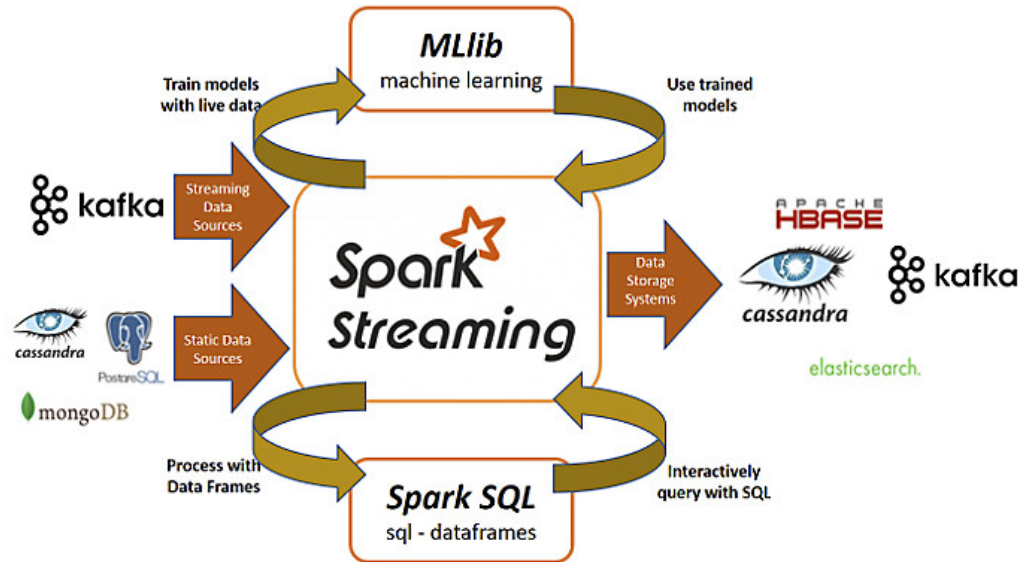
Hình 23: Biểu diễn dữ liệu trong RDD.

- Spark xây dựng đồ thị hướng không chu trình (DAG) để xác định trình tự thực hiện và phụ thuộc giữa các phép toán trên dữ liệu. Dựa vào DAG, Spark chia công việc thành các giai đoạn (stages), mỗi giai đoạn gồm nhiều tác vụ có thể chạy độc lập và đồng thời. Cách tổ chức này giúp tối ưu hoá việc sử dụng tài nguyên và đảm bảo khả năng mở rộng.
- Spark sử dụng in - memory computation để tối ưu xử lý phân tán, tự động quản lý lỗi. Nếu một node bị lỗi, Spark có thể phục hồi từ lineage (tập các thao tác tạo nên RDD đó).



Hình 24: DAG.

5.4 Hệ sinh thái tích hợp của Spark



Hình 25: Hệ sinh thái của Apache Spark.

Apache Spark gồm có 5 thành phần chính : Spark Core, Spark Streaming, Spark SQL, MLlib và GraphX.

- **Spark Core:** là nền tảng cho các thành phần còn lại và các thành phần này muốn được khởi chạy thì phải thông qua Spark Core do nó đảm nhận vai trò thực hiện công việc tính toán và xử lý trong bộ nhớ (In-Memory computing). Spark Core cung cấp các chức năng cơ bản như:

- + *RDD (Resilient Distributed Dataset):* cấu trúc dữ liệu phân tán, bất biến và có khả năng phục hồi sau lỗi.
- + *Quản lý bộ nhớ (In-Memory Computing):* giảm thời gian truy xuất dữ liệu so với Hadoop MapReduce.
- + *Lập lịch tác vụ (Task Scheduling):* phân phối công việc trên các node trong cluster.

Spark Core đóng vai trò xử lý dữ liệu thô từ log server, sensor IOT và xây dựng các ứng dụng phân tán tùy chỉnh.

- **Spark SQL:** cho phép thực hiện truy vấn trên SQL trên dữ liệu phân tán, hỗ trợ:

- + *DataFrame/Dataset API*: làm việc với dữ liệu dạng bảng.
- + *Tích hợp đa nguồn*: đọc, ghi dữ liệu từ Hive, Parquet, JSON, CSV. Spark SQL là một mô-đun Spark để xử lý dữ liệu có cấu trúc. Nó cung cấp một chương trình trừu tượng gọi là DataFrames và cũng có thể hoạt động như một công cụ truy vấn SQL phân tán.

Spark SQL là một mô-đun Spark để xử lý dữ liệu có cấu trúc. Nó cung cấp một chương trình trừu tượng gọi là DataFrames và cũng có thể hoạt động như một công cụ truy vấn SQL phân tán.

- **Spark Streaming**: được sử dụng để thực hiện việc phân tích luồng dữ liệu (streaming data) theo thời gian thực bằng việc coi stream là các mini-batches và thực hiện kỹ thuật RDD transformation đối với các dữ liệu mini-batches này. Qua đó cho phép các đoạn code được viết cho xử lý batch có thể được tận dụng lại vào trong việc xử lý stream, làm cho việc phát triển lambda architecture được dễ dàng hơn. Spark Streaming là thành phần hỗ trợ xử lý các dữ liệu streams, dữ liệu đầu vào có thể là từ nhiều nguồn như Socket, Kafka, Flume,...
- **MLlib (Machine Learning Library)**: MLlib là một nền tảng học máy phân tán bên trên Spark do kiến trúc phân tán dựa trên bộ nhớ. Theo các so sánh benchmark Spark MLlib nhanh hơn 9 lần so với phiên bản chạy trên Hadoop (Apache Mahout). MLlib cung cấp các thư viện hỗ trợ nhiều thuật toán Machine Learning tối ưu cho môi trường phân tán.
- **GraphX**: là nền tảng xử lý đồ thị dựa trên Spark. Nó cung cấp các API để diễn tả các tính toán trong đồ thị bằng cách sử dụng Pregel API. GraphX là API Spark dành cho đồ thị và tính toán song song đồ thị.

5.4.1 DDR

- **Khái niệm**: Resilient Distributed Datasets (RDDs) là một cấu trúc dữ liệu cơ bản và quan trọng trong Apache Spark, nằm trong Spark Core, cho phép xử lý dữ liệu phân tán trên các cụm máy tính một cách linh hoạt và hiệu quả. Đây là một phần chính của nền tảng Apache Spark, giúp đơn giản hóa việc xử lý dữ liệu lớn bằng cách tự động xử lý phân tán và chịu sự cố.

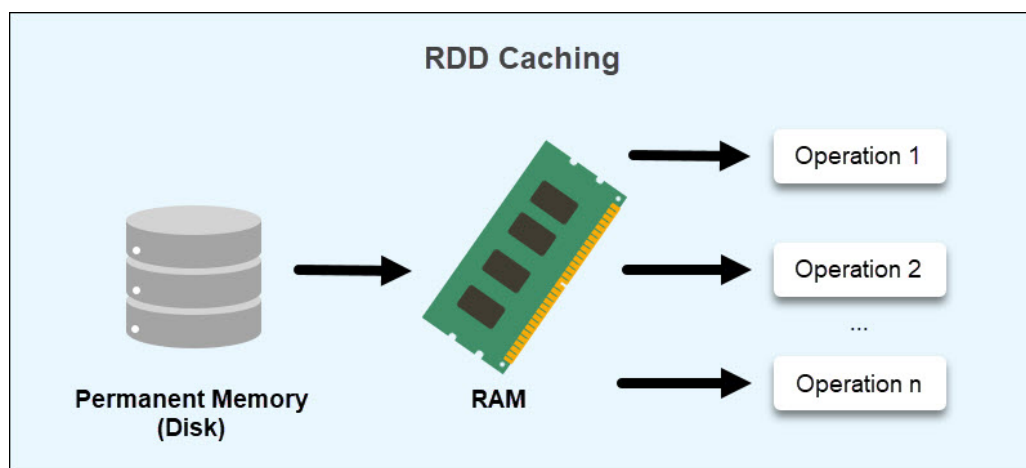
- RDD đại diện cho:

- + *Phân tán (Distributed)*: dữ liệu được chia thành các partition và phân phối trên nhiều node.
- + *Bất biến (Immutable)*: không thể thay đổi sau khi tạo, chỉ có thể tạo RDD mới từ phép biến đổi (transformation).
- + *Tự phục hồi (Resilient)*: có khả năng tái tính toán khi mất dữ liệu nhờ cơ chế lineage.

- Tại sao cần RDD:

RDD giải quyết những thiếu sót của MapReduce trong việc chia sẻ dữ liệu. Khi sử dụng lại dữ liệu để tính toán, MapReduce yêu cầu ghi vào bộ lưu trữ ngoài (HDFS, Cassandra...). Các quy trình đọc và ghi giữa các công việc tiêu tốn một lượng bộ nhớ đáng kể. Hơn nữa, việc chia sẻ dữ liệu giữa các tác vụ diễn ra chậm do phải sao chép, tuần tự hóa và tăng dung lượng đĩa sử dụng.

RDD nhằm mục đích giảm việc sử dụng hệ thống lưu trữ ngoài bằng cách tận dụng lưu trữ hoạt động tính toán trong bộ nhớ. Cách tiếp cận này cải thiện tốc độ trao đổi dữ liệu giữa các tác vụ từ 10 đến 100 lần. Tốc độ là yếu tố quan trọng khi làm việc với khối lượng dữ liệu lớn. Spark giúp đào tạo machine learning algorithms và xử lý khối lượng lớn dữ liệu để phân tích dễ dàng hơn.



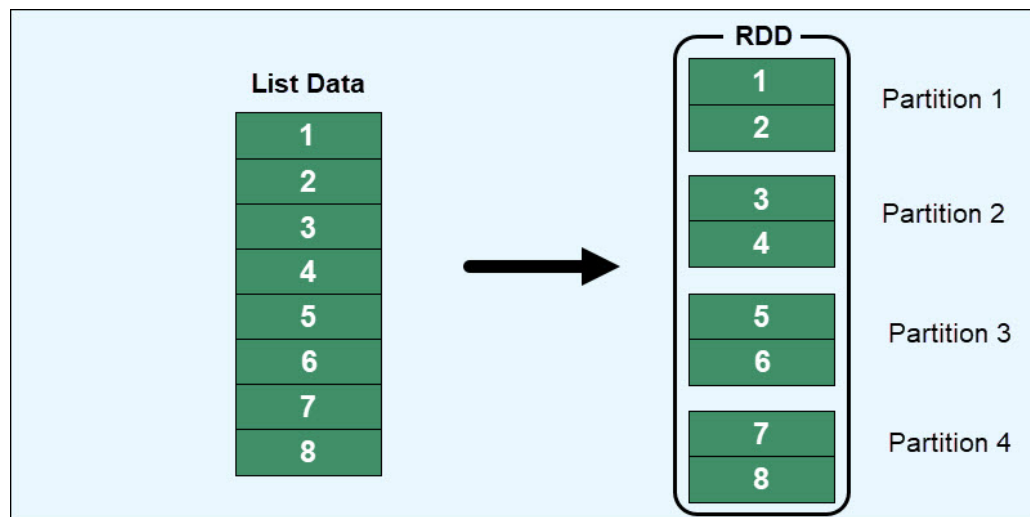
Hình 26: Cơ chế lưu trữ đệm (Caching) của RDD trong Spark.

- RDD lưu trữ như thế nào:

RDD lưu trữ dữ liệu ở chế độ chỉ đọc, khiến dữ liệu không thể thay đổi. Thực hiện các thao tác trên RDD hiện có sẽ tạo ra các đối tượng mới mà không cần thao tác dữ liệu hiện có.

RDD nằm trong RAM thông qua quá trình lưu trữ đệm. Dữ liệu không phù hợp sẽ được tính toán lại để giảm kích thước hoặc được lưu trữ trên bộ nhớ lưu trữ cố định. Lưu trữ đệm cho phép truy xuất dữ liệu mà không cần đọc từ đĩa, giảm chi phí đĩa.

RDD phân phối thêm lưu trữ dữ liệu trên nhiều phân vùng. Phân vùng cho phép khôi phục dữ liệu trong trường hợp một nút bị lỗi và đảm bảo dữ liệu luôn khả dụng.



Hình 27: RDD chia nhỏ dữ liệu để xử lý song song.

RDD của Spark sử dụng persistence optimization technique (kỹ thuật tối ưu hóa tính bền bỉ) để lưu kết quả tính toán. Hai phương pháp giúp đạt được tính bền bỉ của RDD:

- + cache()
- + persist()

Các phương pháp này cung cấp cơ chế lưu trữ tương tác bằng cách chọn các mức lưu trữ khác nhau. Bộ nhớ đệm có khả năng chịu lỗi, cho phép tạo lại các phân vùng RDD bị mất thông qua các hoạt động tạo ban đầu.

- Tính năng của Spark RDD:

In-Memory Computation (Tính toán trong bộ nhớ): Trong tính toán trong bộ nhớ, dữ liệu được lưu trong bộ nhớ truy cập ngẫu nhiên (RAM) thay vì một ổ đĩa chậm và được xử lý song song. Sử dụng điều này, chúng ta có thể phát hiện một mẫu, phân tích dữ liệu lớn. Điều này đã trở nên phổ biến vì nó làm giảm chi phí bộ nhớ. Tính toán dữ liệu nằm trong bộ nhớ để truy cập nhanh hơn và ít thao tác I/O hơn.

Fault tolerance (Khả năng chịu lỗi): Spark hoạt động trên dữ liệu trong các hệ thống tệp chịu lỗi như HDFS hoặc S3 . Vì vậy, tất cả các RDD được tạo từ dữ liệu chịu lỗi đều có khả năng chịu lỗi. Song điều này không đúng đối với dữ liệu phát trực tuyến/trực tiếp (dữ liệu qua mạng). Vì vậy, nhu cầu chính của khả năng chịu lỗi trong Spark là đối với loại dữ liệu này. Nghĩa chịu lỗi cơ bản của Spark là:

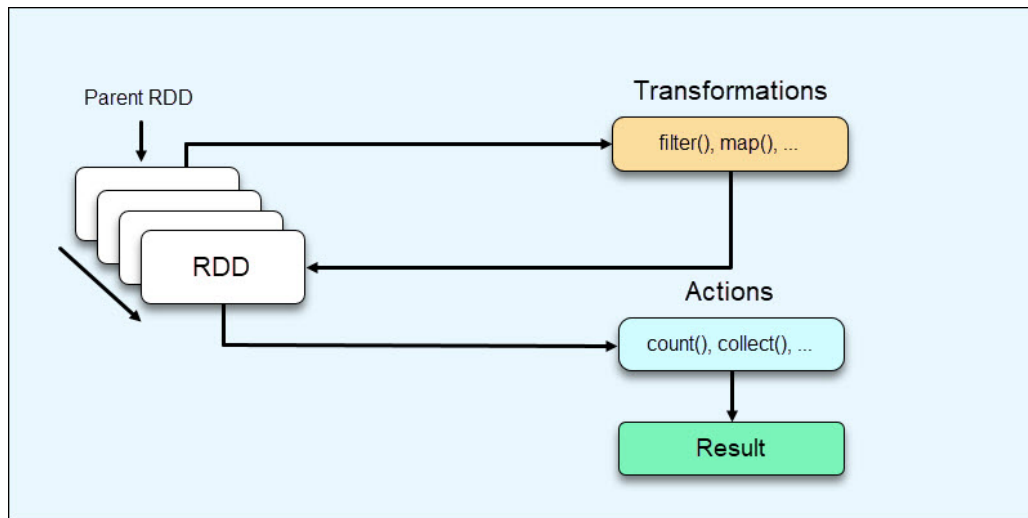
- + Vì Apache Spark RDD là một tập dữ liệu không thay đổi cho nên mỗi Spark RDD sẽ ghi nhớ nguồn gốc của hoạt động xác định được sử dụng trên tập dữ liệu đầu vào có khả năng chịu lỗi để tạo ra nó.
- + Nếu do lỗi của một nút công nhân mà bất kỳ phân vùng nào của RDD bị mất, thì phân vùng đó có thể được tính toán lại từ tập dữ liệu chịu lỗi ban đầu bằng cách sử dụng dòng dõi các hoạt động.
- + Giả sử rằng tất cả các chuyển đổi RDD đều mang tính xác định, dữ liệu trong RDD được chuyển đổi cuối cùng sẽ luôn giống nhau bất kể lỗi trong cụm Spark.

Immutability (Tính bất biến): Spark RDD là bất biến, có nghĩa là một khi RDD được tạo ra, nó không thể thay đổi được. Dữ liệu hiện có không thể thay đổi và các transformation (chuyển đổi) trên dữ liệu hiện có sẽ tạo ra RDD mới. Đây là thiết kế và là đặc điểm chính của RDD trong Spark.

Lazy evaluation (Đánh giá lười biếng): Đánh giá Lazy trong Sparks có nghĩa là Spark sẽ không bắt đầu thực hiện quy trình cho đến khi một ACTION được gọi. Chúng ta chỉ thực hiện các chuyển đổi trên khung dữ liệu/bộ dữ liệu/RDD, Spark ít quan tâm nhất. Khi Spark thấy một ACTION được gọi, nó bắt đầu xem xét tất cả các chuyển đổi và tạo DAG. DAG đơn giản là chuỗi các hoạt động cần được thực hiện trong một quy trình để có được đầu ra kết quả. Nếu Spark có thể đợi cho đến khi Action được gọi, nó có thể hợp nhất một số chuyển đổi hoặc bỏ qua một số chuyển đổi không cần thiết và chuẩn bị một kế hoạch thực hiện hoàn hảo.

- **Hoạt động của Spark RDD:**

Spark RDD cung cấp hai loại hoạt động: Transformations và Actions.



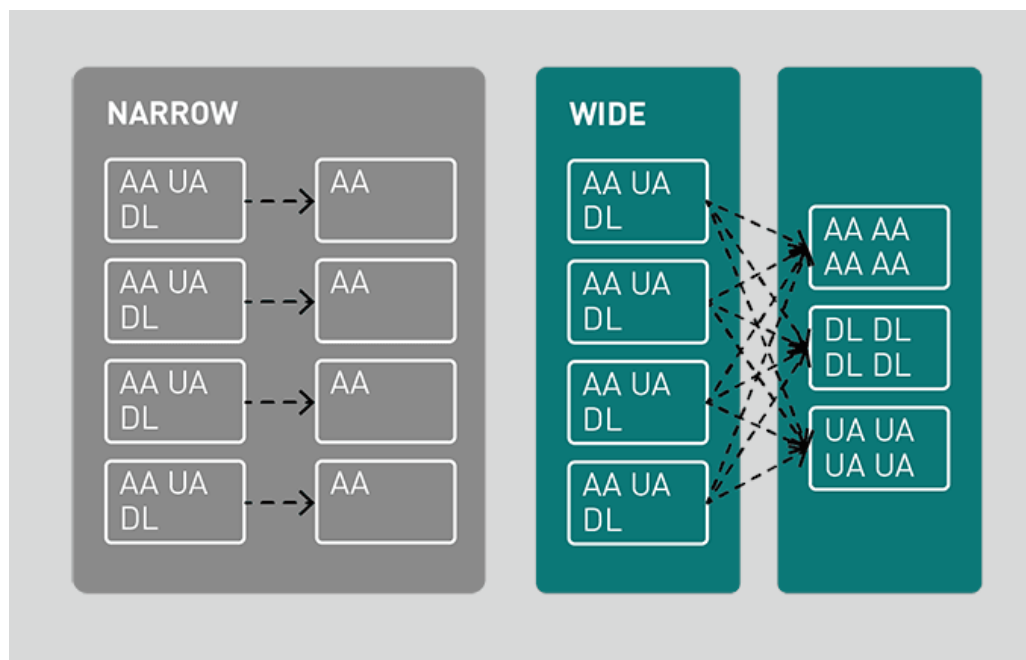
Hình 28: Quy trình hoạt động của RDD trong Apache Spark Transformations và Actions.

Transformation

Spark Transformation là các hoạt động trên RDD dẫn đến việc tạo ra RDD. Mỗi khi ta áp dụng bất kì transform nào, nó lại tạo ra một RDD mới. Do đó, các RDD đầu vào là bất biến vì RDD về bản chất không thay đổi, chỉ là ta thấy một RDD khác là kết quả từ RDD ban đầu.

Có hai loại Transformations:

- + *Narrow Transformations*: Mỗi partition đầu ra chỉ phụ thuộc vào một partition đầu vào duy nhất.
- + *Wide Transformations*: Mỗi partition đầu ra chỉ phụ thuộc vào một partition đầu vào duy nhất.



Hình 29: Narrow Transformations và Wide Transformations.

- Ưu nhược điểm của Spark RDD:

Ưu điểm của việc sử dụng RDD:

- + *Khả năng phục hồi dữ liệu:* Cơ chế tự phục hồi đảm bảo dữ liệu không bao giờ bị mất, bất kể máy có bị hỏng hay không.
- + *Tính nhất quán của dữ liệu:* Vì RDD không thay đổi theo thời gian và chỉ có thể đọc nên tính nhất quán của dữ liệu được duy trì trong suốt các hoạt động khác nhau.
- + *Tốc độ hiệu suất:* Lưu trữ dữ liệu trong RAM bất cứ khi nào có thể thay vì trên đĩa. Tuy nhiên, RDD duy trì khả năng lưu trữ trên đĩa để cung cấp hiệu suất và tính linh hoạt tăng cường đáng kể.

Nhược điểm của việc sử dụng RDD:

- + *Không có chế độ xem sơ đồ dữ liệu:* RDD gặp khó khăn khi xử lý dữ liệu có cấu trúc. Một lựa chọn tốt hơn để xử lý dữ liệu có cấu trúc là thông qua API DataFrames và Datasets, được tích hợp hoàn toàn với RDD trong Spark.
- + *Thu gom rác:* Vì RDD là các đối tượng trong bộ nhớ nên chúng phụ thuộc rất nhiều vào quản lý bộ nhớ và tuần tự hóa của Java. Điều này gây ra giới hạn hiệu suất khi dữ liệu tăng lên.

- + *Sự cố tràn bộ nhớ*: Khi RDD hết RAM, thông tin sẽ nằm trên đĩa, cần thêm RAM và dung lượng đĩa để khắc phục sự cố tràn bộ nhớ.
- + *Không có tối ưu hóa tự động*: RDD không có chức năng tối ưu hóa đầu vào tự động. Trong khi các đối tượng Spark khác, chẳng hạn như DataFrames và Datasets, sử dụng trình tối ưu hóa Catalyst, đối với RDD, việc tối ưu hóa diễn ra thủ công.

5.4.2 Spark SQL

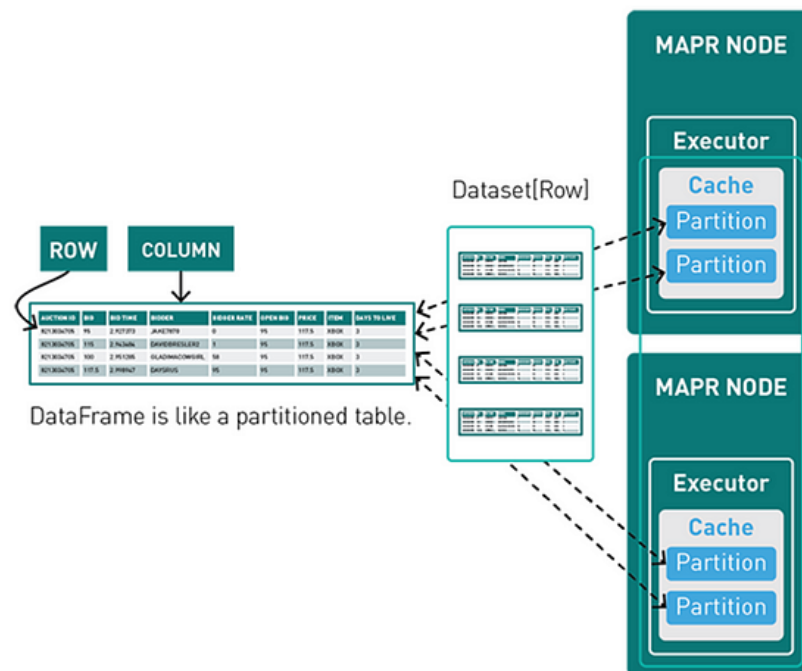
- DataFrame trong Spark:

DataFrame là cấu trúc dữ liệu chính của Spark SQL và là một tập hợp dữ liệu phân tán, không thay đổi được, được tổ chức thành các hàng, trong đó mỗi hàng bao gồm một tập hợp các cột và mỗi cột có một tên và một loại liên quan. Hiểu đơn giản, DataFrame tương tự như một Table trong một cơ sở dữ liệu quan hệ. Có các trường, được định sẵn kiểu dữ liệu và tập hợp các bản ghi. Tuy nhiên, một DataFrame có thể đại diện cho lượng dữ liệu lớn hơn rất nhiều so với các bảng trong DataBase.

DataFrame được tối ưu cho việc xử lý dữ liệu có cấu trúc và có thể được lưu trữ trong bộ nhớ hoặc được đọc từ các nguồn dữ liệu như HDFS, S3, Kafka, Cassandra, JDBC, ...

Các đặc điểm của DataFrame:

- + *Distributed (Phân tán)*: Dữ liệu được tự động chia nhỏ thành các partition trên cluster.
- + *Immutable (Bất biến)*: Không thay đổi được trực tiếp, mọi hành động đều tạo DataFrame mới.
- + *Lazy Evaluation (Đánh giá lười biếng)*: Chỉ thực thi khi gọi action.
- + *Optimized Execution (Tối ưu hóa tự động)*: Tự động tối ưu bằng Catalyst Optimizer.
- + *Hỗ trợ đa ngôn ngữ*: Python, Java, Scala, R.



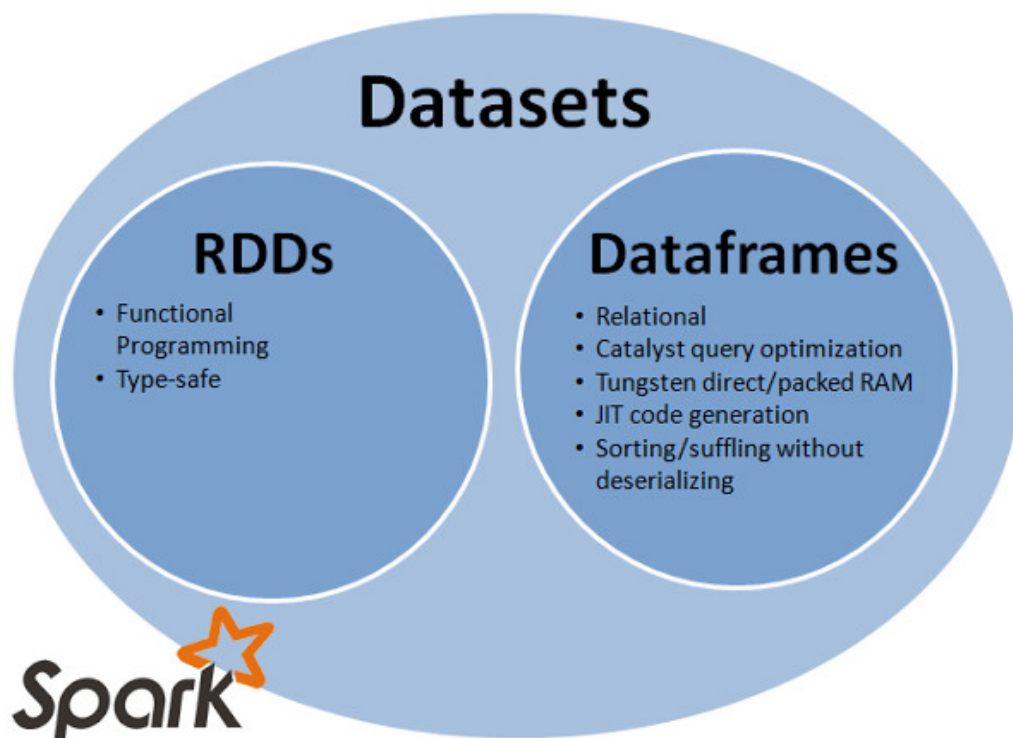
Hình 30: Cấu trúc và phân vùng của DataFrame trong Apache Spark.

DataFrame được xây dựng trên nền tảng RDD nhưng có hiệu suất cao hơn nhờ tối ưu hóa Catalyst Optimizer được tích hợp trong Spark SQL.

- **DataSet trong Spark:** Dataset cũng là một cấu trúc SparkSQL và là phần mở rộng của DataFrame API. Dataset API kết hợp tối ưu hóa hiệu suất của DataFrame và sự tiện lợi của RDD. Ngoài ra, API phù hợp hơn với các ngôn ngữ có kiểu dữ liệu mạnh. Tính an toàn kiểu dữ liệu được cung cấp và giao diện lập trình hướng đối tượng khiến Dataset API chỉ khả dụng cho Java và Scala.

DataSet đi kèm với phong cách OOP và tính năng an toàn thời gian biên dịch thân thiện với nhà phát triển như RDD cũng như các tính năng tăng cường hiệu suất của DataFrame: trình tối ưu hóa Catalyst và quản lý bộ nhớ tùy chỉnh.

DataFrame thực tế được coi là tập dữ liệu của các đối tượng hàng. Tức là mỗi hàng của Dataset là một DataFrame. Vì vậy, chúng ta luôn có thể chuyển đổi qua lại giữa DataFrame và DataSet, tạo sự linh hoạt khi làm việc với nhiều loại dữ liệu khác nhau.



Hình 31: Kiến trúc của DataSet.

Điểm độc đáo của DataSet là được trang bị thêm tính năng Encoders: Cơ chế chuyển đổi giữa các đối tượng JVM (Scala, Java object) và định dạng nhị phân (binary format) mà Spark sử dụng nội bộ. Từ đó DataSet trở nên an toàn về kiểu dữ liệu trong quá trình biên dịch, nhờ đó giúp phát hiện ra lỗi tiềm ẩn trước khi thực thi.

- **Khác biệt quan trọng giữa DataFrame và DataSet:**

Đối tượng Row trong DataFrame

Trong Spark SQL, các bản ghi (rows) trong DataFrame luôn được cố định dưới dạng đối tượng kiểu Row. Điều này có nghĩa là:

- + DataFrame không có kiểm tra kiểu dữ liệu tại thời điểm biên dịch, vì các cột trong DataFrame chỉ được gọi thông qua tên (dạng chuỗi).
- + Việc truy cập vào dữ liệu trong DataFrame dựa trên tên cột (column name). Nếu có sai sót trong quá trình truyền tên cột, ví dụ do gõ sai, lỗi sẽ chỉ được phát hiện khi thực thi mã (Runtime Exception).
- + Việc cố định dạng đối tượng có thể gây ra lỗi trong quá trình thực thi, đặc biệt là khi làm việc với các tập dữ liệu lớn và phức tạp, vì trình biên dịch không có khả năng kiểm tra kiểu dữ liệu hay xác minh tên cột là đúng hay không.

Định nghĩa kiểu dữ liệu tùy chỉnh trong Dataset với case class

Dataset cung cấp tính năng an toàn về kiểu dữ liệu (type-safety) cao hơn so với DataFrame nhờ sử dụng case class để định nghĩa kiểu dữ liệu của từng bản ghi. Điều này mang lại một số lợi ích quan trọng:

- + Với Dataset, mỗi bản ghi có thể được định nghĩa bằng một case class cụ thể (ví dụ: `Person(id: Int, name: String, age: Int)`). Điều này giúp đảm bảo rằng kiểu dữ liệu của các trường là cố định và được kiểm tra tại thời điểm biên dịch (compile-time).
- + Việc truy cập dữ liệu trong Dataset sử dụng các thành viên của case class (dạng `.field`), thay vì dựa trên tên cột dạng chuỗi như trong DataFrame. Điều này giúp giảm thiểu các lỗi do gõ sai tên cột.
- + Trình biên dịch có thể phát hiện lỗi về kiểu dữ liệu trong Dataset trước khi thực thi, giúp tránh được các lỗi runtime mà DataFrame có thể gặp phải.

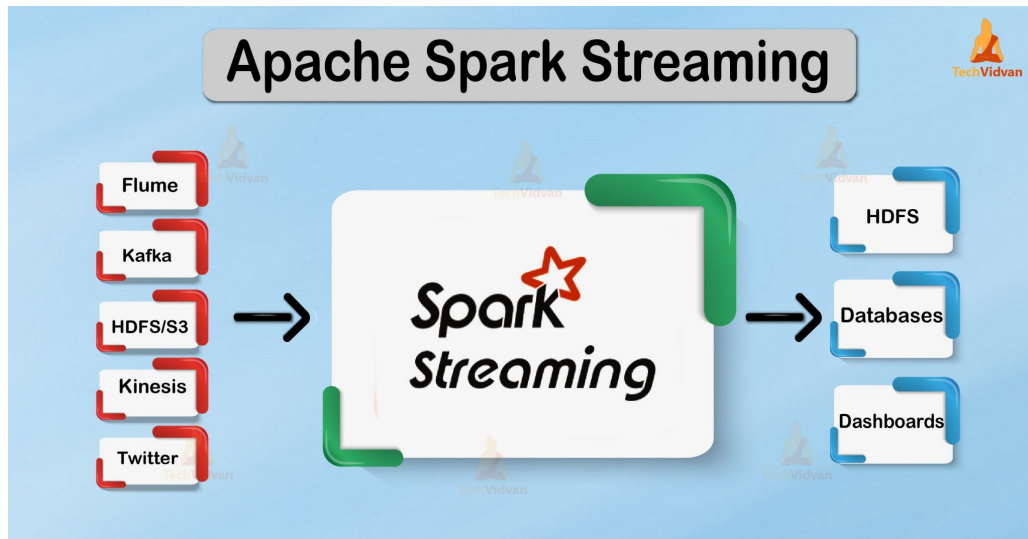
5.4.3 Spark Streaming

Spark Streaming cho phép xử lý dòng dữ liệu thời gian thực theo mô hình micro-batch. Dữ liệu được chia nhỏ thành các lô (batch) ngắn, sau đó xử lý như dữ liệu tĩnh bằng Spark Core.

DStream trong Spark Streaming:

- Discreted Stream (DStream), khái niệm trừu tượng cơ bản trong Spark Streaming, là một chuỗi liên tục các RDD (cùng loại) biểu diễn một luồng dữ liệu liên tục. DStream có thể được tạo từ dữ liệu trực tiếp (chẳng hạn như dữ liệu từ HDFS, Kafka hoặc Flume).
- Cơ chế hoạt động: Dữ liệu thời gian thực được chia thành các batch nhỏ (thường mỗi batch là 1 – vài giây). Mỗi batch là một RDD được xử lý bởi Spark Core. Điều này cho phép sử dụng toàn bộ sức mạnh xử lý phân tán của Spark Core để xử lý dữ liệu streaming.
- DStream được xây dựng trên nền tảng RDD nên có thể áp dụng các transformations giống như trên RDD. DStream cũng có thể tự động khôi phục dữ liệu bị mất hoặc xử lý lại batch trong trường hợp task/node bị lỗi.

- Có thể đọc dữ liệu từ nhiều nguồn streaming phổ biến như: Apache Kafka, Apache Flume, Socket TCP, Kinesis, HDFS, S3,... Kết quả cũng có thể được ghi ra HDFS, S3, data dashboard hoặc hệ thống các file hệ thống khác.



Hình 32: Apache Spark Streaming.

Dữ liệu từ DStream có thể dễ dàng chuyển đổi sang DataFrame để thực hiện truy vấn SQL hoặc sử dụng các mô hình học máy từ MLlib.

5.4.4 Apache Spark MLlib

- Giới thiệu:

Apache Spark MLlib (Machine Learning Library) là thư viện học máy của Apache Spark, được thiết kế để thực hiện các thuật toán học máy một cách hiệu quả, phân tán và mở rộng quy mô dễ dàng. MLlib tích hợp chặt chẽ với các thành phần khác của Spark, giúp việc xử lý dữ liệu, phân tích và triển khai các mô hình học máy trở nên nhanh chóng.

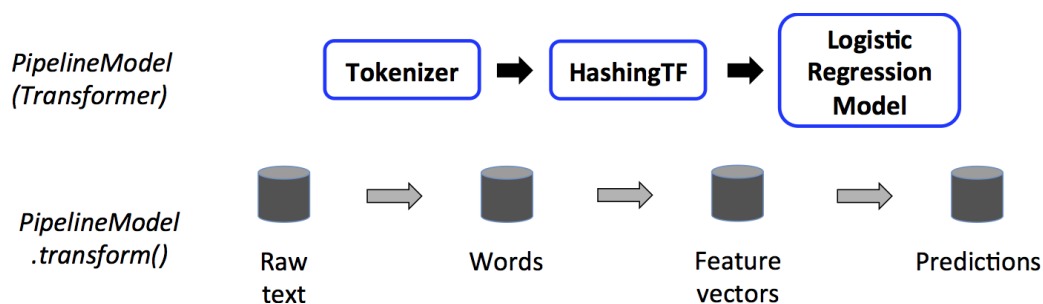
- Đặc điểm nổi bật của Spark MLlib:

- + *Hiệu năng cao:* Tận dụng khả năng tính toán phân tán và xử lý trong bộ nhớ của Spark để tăng tốc huấn luyện mô hình.
- + *Tích hợp chặt chẽ với hệ sinh thái Spark:* Cung cấp API cấp cao cho Python, Java, Scala và R.

- + *Hỗ trợ đa ngôn ngữ*: Cung cấp API cấp cao cho Python, Java, Scala và R.
- + *Thư viện thuật toán đa dạng*: Hỗ trợ nhiều thuật toán học có giám sát, không giám sát, và các công cụ xử lý đặc trưng (feature engineering)
- **Một số thuật toán trong MLlib:**
 - + *Thuật toán gợi ý (Collaborative Filtering)*: Alternating Least Squares (ALS)
 - + *Hồi quy (Regression)*: Linear Regression, Lasso, Ridge, Logistic Regression, Support Vector Machines (SVM)
 - + *Phân cụm (Clustering)*: K-Means, Gaussian Mixture
 - + *Phân loại (Classification)*: Decision Trees, Naive Bayes, Random Forest
 - + *Giảm chiều dữ liệu (Dimensionality Reduction)*: PCA, SVD
- **Thành phần chính trong Pipeline học máy của Spark:**

MLlib cung cấp mô hình Pipeline tương tự scikit-learn, bao gồm các thành phần sau:

- + *DataFrame*: Dữ liệu được tổ chức dưới dạng bảng, sử dụng từ Spark SQL.
- + *Transformer*: Biến đổi DataFrame này thành DataFrame khác (ví dụ: HashingTF, Binarizer, mô hình đã huấn luyện).
- + *Estimator*: Thuật toán học máy để huấn luyện mô hình từ dữ liệu (ví dụ: LogisticRegression, DecisionTreeClassifier).
- + *Pipeline*: Chuỗi các Transformer và Estimator để xây dựng quy trình học máy đầy đủ.
- + *Parameter*: API thống nhất cho việc thiết lập siêu tham số của Estimator và Transformer.



Hình 33: PipelineModel.

Ví dụ minh họa: Quy trình học máy sử dụng Pipeline. Gồm 3 bước:

- + *Tokenizer*: Nhận đầu vào là văn bản thô (raw text), tách thành các từ.
- + *HashingTF*: Chuyển đổi các từ thành vector đặc trưng (feature vectors).
- + *Logistic Regression Model*: Áp dụng mô hình hồi quy logistic để đưa ra dự đoán (predictions).

- Thư viện học máy tích hợp với Spark MLlib:

Apache Spark MLlib có khả năng tích hợp tốt với nhiều thư viện học máy ngoài Spark, giúp mở rộng khả năng của nó trong các dự án phức tạp. Một số thư viện tiêu biểu bao gồm:

- + *XGBoost4J-Spark*: Kết hợp giữa XGBoost và Spark để xử lý mô hình boosting trên dữ liệu lớn.
- + *scikit-learn*: Dùng cho các bước xử lý trước dữ liệu hoặc đánh giá mô hình, thông qua chuyển đổi dữ liệu giữa Spark DataFrame và Pandas.
- + *TensorFlow, PyTorch*: Dùng để huấn luyện hoặc triển khai các mô hình học sâu trên Spark, thường kết hợp thông qua Spark TensorFlow Distributor hoặc Petastorm.

- Cách thức hoạt động của Spark MLlib:

MLlib hoạt động theo mô hình xử lý phân tán trên Spark, dựa vào Spark Core và Spark SQL. Quy trình tổng thể như sau:

- + *Tiền xử lý dữ liệu (Data Preprocessing)*: Nhập dữ liệu vào Spark dưới dạng DataFrame. Dùng các Transformer như StringIndexer, OneHotEncoder, VectorAssembler để xử lý đặc trưng.
- + *Xây dựng Pipeline*: Kết hợp các bước tiền xử lý và mô hình học máy thành một pipeline. Pipeline này có thể huấn luyện (fit) và dự đoán (transform) trên toàn bộ tập dữ liệu.
- + *Huấn luyện mô hình*: Dữ liệu được chia nhỏ thành các phân vùng và xử lý song song trên các node của cụm Spark. Sử dụng thuật toán phân tán để huấn luyện mô hình trên từng phân vùng.
- + *Đánh giá mô hình*: Chọn mô hình tối ưu.

- + *Lưu mô hình và triển khai*: Mô hình và Pipeline có thể được lưu vào HDFS hoặc local và triển khai trên môi trường phân tán hoặc thời gian thực.

5.5 Ưu điểm và nhược điểm của Apache Spark:

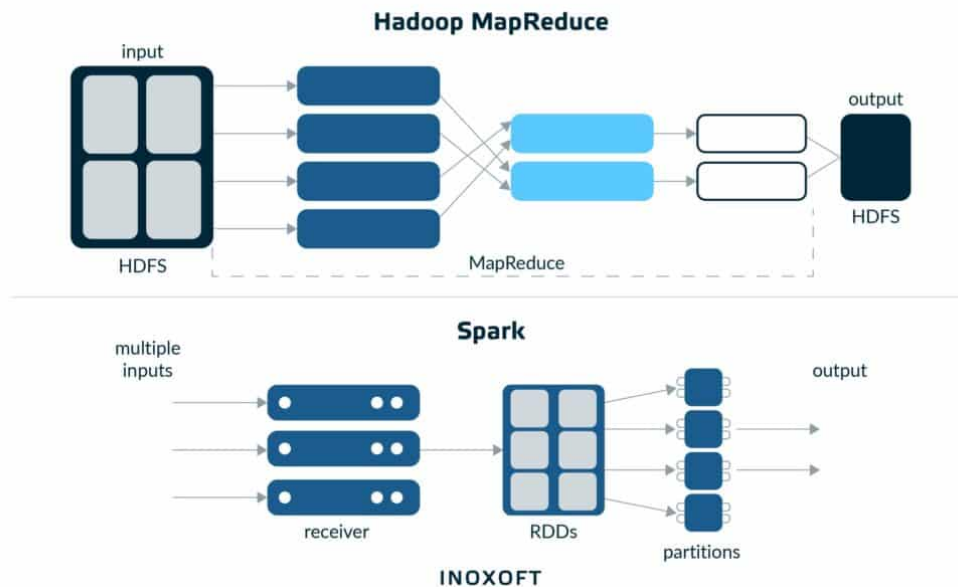
- Ưu điểm:

- + *Tốc độ xử lý cao, xử lý dữ liệu phân tán hiệu quả*: Spark nổi bật với khả năng xử lý dữ liệu cực nhanh nhờ cơ chế xử lý trong bộ nhớ (in-memory processing), giúp tăng tốc độ phân tích dữ liệu gấp nhiều lần so với Hadoop MapReduce.
- + *Hỗ trợ đa ngôn ngữ lập trình*: Spark hỗ trợ nhiều ngôn ngữ lập trình phổ biến như Scala, Java, Python và R, giúp các nhà phát triển dễ dàng tích hợp và phát triển ứng dụng theo sở trường.
- + *Xử lý đa dạng khối lượng công việc*: Spark có thể xử lý nhiều loại tác vụ khác nhau như xử lý theo lô, xử lý thời gian thực, học máy và xử lý đồ thị, tất cả trong cùng một nền tảng duy nhất.
- + *Hỗ trợ truy vấn SQL*: Spark SQL cho phép người dùng truy vấn dữ liệu bằng ngôn ngữ SQL, rất hữu ích cho việc xử lý và phân tích dữ liệu có cấu trúc.
- + *Thư viện phong phú*: Có nhiều thư viện dành cho xử lý dữ liệu lớn, Spark tích hợp MLlib – thư viện mạnh mẽ cho các thuật toán học máy, giúp triển khai nhanh các mô hình phân tích và dự đoán.
- + *Cộng đồng lớn và phát triển mạnh*: Spark được hỗ trợ bởi cộng đồng người dùng đông đảo, liên tục cập nhật và cải tiến, mang lại sự ổn định và phát triển bền vững cho nền tảng.

- Nhược điểm:

- + *Tiêu tốn tài nguyên RAM*: Việc xử lý dữ liệu trong bộ nhớ khiến Spark đòi hỏi hệ thống phần cứng mạnh, đặc biệt là RAM lớn, dẫn đến chi phí vận hành cao.
- + *Không có hệ thống lưu trữ riêng*: Spark không tích hợp hệ thống lưu trữ dữ liệu bền vững như Hadoop HDFS, nên cần kết hợp với các hệ thống lưu trữ khác như HDFS, Cassandra hoặc Amazon S3 để đảm bảo an toàn dữ liệu.
- + *Nguy cơ mất dữ liệu khi cấu hình lỗi*: Nếu không cấu hình lưu trữ đúng cách, dữ liệu có thể bị mất khi xảy ra sự cố hệ thống, vì Spark chủ yếu xử lý trên bộ nhớ tạm thời.

5.5.1 Sự khác biệt giữa Apache Hadoop và Apache Spark



Hình 34: Sự khác biệt giữa Hadoop và Spark.

Tiêu chí	Apache Hadoop	Apache Spark
Kiến trúc	Hadoop lưu trữ và xử lý dữ liệu trên kho lưu trữ bên ngoài.	Spark lưu trữ và xử lý dữ liệu trên bộ nhớ trong, dựa trên RDD.
Cơ chế xử lý	Theo lô (batch), qua MapReduce.	Theo lô và theo thời gian thực (real - time).
Hiệu suất	Chậm hơn do đọc/ghi từ đĩa.	Nhanh hơn nhờ lưu trữ trong RAM.
Chi phí	Hadoop có giá hợp lý.	Spark có giá đắt hơn do yêu cầu nhiều RAM.
Máy học	Hadoop tích hợp các thư viện bên ngoài để cung cấp các tính năng máy học.	Spark đã tích hợp sẵn các thư viện máy học.

Bảo mật	Có tính năng bảo mật mạnh mẽ, mã hóa lưu trữ và kiểm soát truy cập.	Có khả năng bảo mật cơ bản. Vậy nên bạn cần thiết lập một môi trường hoạt động an toàn cho việc triển khai Spark.
----------------	---	---

Bảng 3: So sánh Hadoop với Spark

5.6 Ứng dụng của Apache Spark

- **Tài chính:** Apache Spark đóng vai trò quan trọng trong ngành tài chính, đặc biệt là ngân hàng và đầu tư. Trong ngân hàng, Spark hỗ trợ dự đoán khách hàng rời bỏ (churn prediction) dựa trên hành vi sử dụng dịch vụ, từ đó giúp đưa ra các biện pháp giữ chân phù hợp. Trong lĩnh vực đầu tư, Spark được dùng để phân tích biến động thị trường và dự đoán giá cổ phiếu nhờ khả năng xử lý big data kết hợp với các thuật toán máy học, giúp nhà đầu tư đưa ra quyết định chính xác hơn.
- **Chăm sóc sức khỏe:** Spark giúp cải thiện hiệu quả trong chăm sóc bệnh nhân bằng cách xây dựng hệ thống quản lý hồ sơ bệnh án toàn diện, theo dõi chỉ số sức khỏe theo thời gian thực. Đồng thời, Spark còn hỗ trợ phân tích dữ liệu y tế để đề xuất phác đồ điều trị tối ưu, giúp nâng cao chất lượng dịch vụ và tối ưu hóa quy trình y tế.
- **Sản xuất:** Trong công nghiệp sản xuất, Spark hỗ trợ bảo trì phòng ngừa bằng cách phân tích dữ liệu hoạt động của thiết bị để dự đoán thời điểm xảy ra sự cố. Điều này giúp giảm thời gian ngừng máy, tăng hiệu suất và giảm chi phí bảo trì thông qua các quyết định bảo trì chính xác và kịp thời.
- **Bán lẻ:** Spark là công cụ đắc lực trong việc cá nhân hóa trải nghiệm mua sắm. Bằng cách phân tích hành vi người dùng, lịch sử mua hàng và phản hồi khách hàng, Spark giúp doanh nghiệp xây dựng hồ sơ khách hàng chi tiết, từ đó đưa ra đề xuất sản phẩm và chiến lược tiếp thị phù hợp. Ngoài ra, Spark còn giúp tự động tùy chỉnh giao diện ứng dụng, website hay cửa hàng theo từng khách hàng, góp phần nâng cao sự hài lòng và trung thành.

5.7 Kết luận

Apache Spark là một trong những nền tảng xử lý dữ liệu lớn được ưa chuộng và sử dụng rộng rãi nhất hiện nay nhờ vào kiến trúc hiện đại, tốc độ xử lý cao và khả năng linh hoạt trong nhiều tác vụ khác nhau. Không giống như các hệ thống cũ như Hadoop MapReduce chỉ hỗ trợ xử lý theo lô, Spark hỗ trợ cả xử lý theo lô và thời gian thực, tận dụng bộ nhớ RAM để tăng hiệu suất đáng kể. Điều này đặc biệt quan trọng trong bối cảnh hiện tại, khi nhu cầu xử lý dữ liệu nhanh chóng, tức thời đang ngày càng gia tăng.

Bên cạnh hiệu năng vượt trội, Spark còn nổi bật nhờ vào hệ sinh thái phong phú, tích hợp sẵn các thư viện cho máy học (MLlib), xử lý đồ thị (GraphX), xử lý dòng dữ liệu (Spark Streaming) và khả năng truy vấn dữ liệu có cấu trúc qua Spark SQL. Tất cả đều hoạt động trên cùng một nền tảng, giúp giảm độ phức tạp và tối ưu hiệu quả triển khai cho các hệ thống phân tích dữ liệu lớn.

Với sự hỗ trợ đa ngôn ngữ (Scala, Python, Java, R), Spark thân thiện với cả nhà phát triển và nhà phân tích dữ liệu, đồng thời có thể dễ dàng tích hợp với các hệ thống lưu trữ dữ liệu khác như Hadoop HDFS, Amazon S3, hoặc hệ thống dữ liệu nội bộ.

Sự phát triển nhanh chóng của Spark cũng cho thấy xu hướng chuyển dịch từ mô hình “Hadoop Stack” sang “Big Data Stack” linh hoạt hơn, nơi Spark đóng vai trò trung tâm trong việc xử lý và khai thác dữ liệu. Điều này mở ra nhiều cơ hội mới cho các tổ chức trong việc tận dụng dữ liệu để ra quyết định, dự báo và cải tiến hoạt động một cách hiệu quả và nhanh chóng hơn.

Tóm lại, Apache Spark không chỉ là một công cụ xử lý dữ liệu đơn thuần, mà còn là một phần quan trọng trong chiến lược chuyển đổi số và phát triển các ứng dụng dựa trên dữ liệu ở thời đại hiện nay.

6 Kết luận và định hướng phát triển

6.1 Tóm tắt lại nội dung đã trình bày

Báo cáo đã trình bày ba nội dung chính liên quan đến xu hướng công nghệ dữ liệu hiện nay, bao gồm: Big Data, NoSQL và Apache Spark. Trước hết, Big Data là khái niệm chỉ tập hợp dữ liệu có khối lượng lớn, tốc độ sinh dữ liệu cao và đa dạng về loại hình. Big Data được đặc trưng bởi mô hình 5V (Volume, Velocity, Variety, Veracity, Value). Các phương pháp xử lý dữ liệu lớn phổ biến hiện nay bao gồm xử lý theo lô (batch), xử lý thời gian thực (stream) và hyper processing. Big Data đã được ứng dụng rộng rãi trong nhiều lĩnh vực như tài chính, y tế, giáo dục, giao thông và chăm sóc khách hàng.

Tiếp theo, NoSQL là mô hình cơ sở dữ liệu phi quan hệ ra đời nhằm khắc phục những hạn chế của hệ quản trị cơ sở dữ liệu quan hệ truyền thống trong việc xử lý dữ liệu phi cấu trúc và quy mô lớn. Các loại NoSQL phổ biến gồm có: Key-Value Store, Document Store, Column-Family Store và Graph Database. NoSQL có ưu điểm linh hoạt, dễ mở rộng và hiệu quả cao trong môi trường dữ liệu đa dạng.

Cuối cùng là Apache Spark – nền tảng xử lý dữ liệu phân tán có hiệu năng cao, đang dần thay thế Hadoop MapReduce nhờ khả năng xử lý nhanh chóng, hỗ trợ lưu trữ trong bộ nhớ và xử lý thời gian thực. Apache Spark gồm nhiều thành phần như Spark Core, Spark SQL, Spark Streaming, GraphX và MLlib. Đặc biệt, Spark MLlib là thư viện học máy tích hợp sẵn, hỗ trợ nhiều thuật toán học máy từ cơ bản đến nâng cao, giúp xây dựng pipeline học máy hoàn chỉnh và có khả năng mở rộng.

6.2 Hướng phát triển trong tương lai

Trong tương lai, Big Data sẽ tiếp tục phát triển mạnh mẽ cùng với sự bùng nổ của Internet of Things (IoT) và trí tuệ nhân tạo (AI). Việc cá nhân hóa trải nghiệm người dùng, tối ưu hóa hoạt động doanh nghiệp, cũng như đảm bảo an toàn và bảo mật dữ liệu sẽ là những yêu cầu trọng tâm. Bên cạnh đó, các nền tảng như Apache Spark cũng sẽ tiếp tục mở rộng, hỗ trợ tốt hơn cho xử lý thời gian thực, tích hợp với các thư viện học sâu như TensorFlow, PyTorch, và tăng khả năng triển khai trên các nền tảng đám mây.

lớn như Azure, AWS, GCP. Đây sẽ là tiền đề quan trọng để xây dựng các hệ thống dữ liệu thông minh, phân tán và hiệu quả hơn trong tương lai.

7 Ứng dụng thực tế

7.1 Dữ liệu

Bộ dữ liệu được sử dụng trong phân tích là NYC TLC Trip Record Data, bao gồm thông tin chi tiết về các chuyến đi của taxi vàng (Yellow Taxi) tại thành phố New York. Dữ liệu do các nhà cung cấp công nghệ (TPEP) ghi nhận và báo cáo cho Ủy ban Taxi và Limousine thành phố New York (TLC), trong đó ghi lại nhiều thuộc tính quan trọng của mỗi chuyến đi như: thời gian, địa điểm, khoảng cách, giá cước và các loại phụ phí.

Nguồn dữ liệu: NYC Taxi and Limousine Commission [1]

Thông tin trong bộ dữ liệu bao gồm:

Trường	Mô tả
tpep_pickup_datetime	Thời gian đồng hồ tính cước bắt đầu.
tpep_dropoff_datetime	Thời gian đồng hồ tính cước kết thúc.
passenger_count	Số lượng hành khách trong chuyến đi.
trip_distance	Quãng đường di chuyển (tính bằng dặm).
RatecodeID	Loại mã cước: 1 = chuẩn, 2 = JFK, 3 = Newark, 4 = Nassau/Westchester, 5 = thương lượng, 6 = đi nhóm, 99 = không xác định.
PULocationID	Mã vùng đón khách (theo vùng taxi NYC).
DOLocationID	Mã vùng trả khách (theo vùng taxi NYC).
payment_type	Phương thức thanh toán: 1 = Thẻ tín dụng, 2 = Tiền mặt, 3 = Không tính phí, 4 = Tranh chấp, 5 = Không rõ, 6 = Hủy giao dịch.
fare_amount	Cước phí chính (tính theo thời gian và khoảng cách).
extra	Phụ phí bổ sung khác (phí ban đêm, đặt trước, v.v).
mta_tax	Thuế MTA.
tip_amount	Tiền boa (tip).

tolls_amount	Tổng phí cầu đường.
congestion_surcharge	Phụ phí tắc đường.
airport_fee	Phí sân bay (chỉ áp dụng ở LaGuardia và JFK).
cbd_congestion_fee	Phí tắc đường khu vực trung tâm NYC (từ 2025-01-05).
VendorID	Mã nhà cung cấp dữ liệu (ví dụ: Curb Mobility, Helix,...).
store_and_fwd_flag	Có lưu trữ cục bộ trước khi gửi không: Y = có, N = không.

Bảng 4: Data Schema of the NYC Taxi and Limousine Commission Dataset

Mục tiêu:

- Hiểu xu hướng doanh thu, lượng chuyển đi theo thời gian, theo ngày lễ và cuối tuần.
- Dự đoán tổng số tiền phải trả cho chuyển đi dựa trên đặc trưng: quãng đường, số hành khách, vùng đón, giờ đón, ngày lễ, v.v.

7.2 Xử lý cơ sở dữ liệu với PySpark

Dữ liệu gốc được lưu trữ dưới định dạng Parquet trên HDFS. Quá trình xử lý được thực hiện hoàn toàn với PySpark, tận dụng khả năng phân tán của Spark để xử lý dữ liệu lớn hiệu quả.

Dữ liệu sau khi xử lý được lưu dưới dạng CSV, với dung lượng khoảng **9GB**. Quá trình xử lý và chuyển đổi dữ liệu đều được thực hiện hoàn toàn bằng **PySpark**, giúp tối ưu hiệu năng trên dữ liệu lớn. [2]

```

root
|-- VendorID: integer (nullable = true)
|-- tpep_pickup_datetime: timestamp (nullable = true)
|-- tpep_dropoff_datetime: timestamp (nullable = true)
|-- passenger_count: double (nullable = true)
|-- trip_distance: double (nullable = true)
|-- RatecodeID: double (nullable = true)
|-- store_and_fwd_flag: string (nullable = true)
|-- PULocationID: integer (nullable = true)
|-- DOLocationID: integer (nullable = true)
|-- payment_type: integer (nullable = true)
|-- fare_amount: double (nullable = true)
|-- extra: double (nullable = true)
|-- mta_tax: double (nullable = true)
|-- tip_amount: double (nullable = true)
|-- tolls_amount: double (nullable = true)
|-- improvement_surcharge: double (nullable = true)
|-- total_amount: double (nullable = true)
|-- congestion_surcharge: double (nullable = true)
|-- Airport_fee: double (nullable = true)

```

Hình 35: Schema.

7.2.1 Tiền xử lý dữ liệu:

Làm sạch dữ liệu: Loại bỏ các dòng thiếu thông tin, sai định dạng thời gian, hoặc chứa các giá trị bất thường như: passenger_count, trip_distance, RatecodeID, store_and_fwd_flag, congestion_surcharge, airport_fee

```

-----+-----+-----+-----+-----+-----+
time|passenger_count|trip_distance|RatecodeID|store_and_fwd_flag|P
-----+-----+-----+-----+-----+
0|          5586198|          0|    5586198|          5586198|
-----+-----+-----+-----+-----+

```

Hình 36: Dữ liệu chứa giá trị bất thường-1.

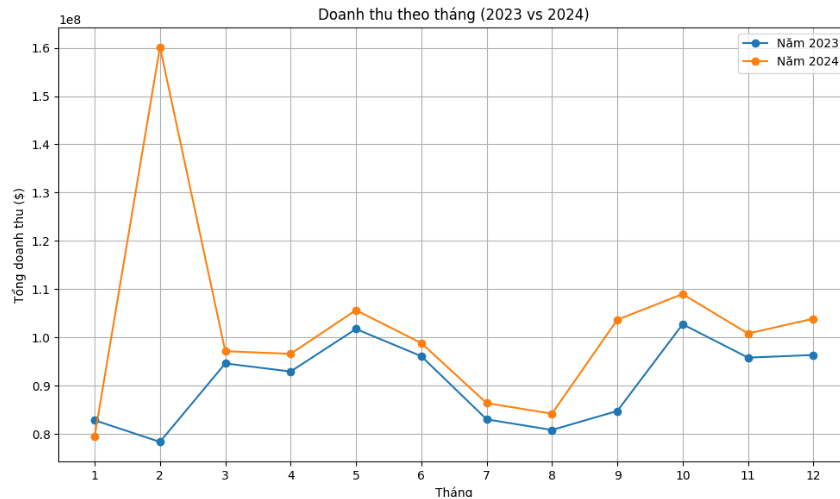
```

--+-----+-----+-----+
nt|congestion_surcharge|Airport_fee|
--+-----+-----+-----+
0|          5586198|          5586198|
--+-----+-----+-----+

```

Hình 37: Dữ liệu chưa giá trị bất thường-2.

Sau khi xử lý và làm sạch dữ liệu, chúng em đã thực hiện phân tích doanh thu theo tháng cho các năm 2023 và 2024. Biểu đồ doanh thu theo tháng cho thấy rõ sự biến động trong các tháng cao điểm như mùa hè và lễ Tết. Năm 2024 có xu hướng tăng trưởng về doanh thu so với năm 2023.



Hình 38: Doanh thu theo tháng năm 2023 và 2024.

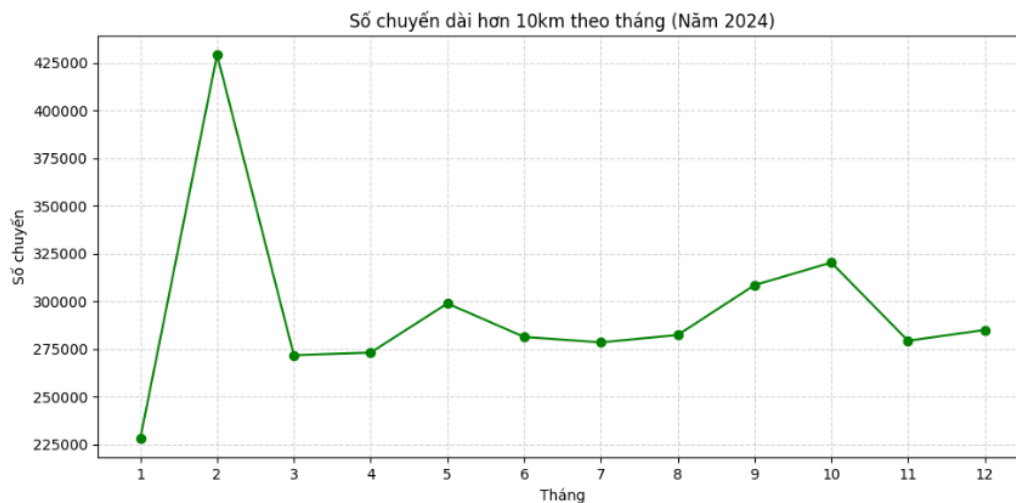
```

+----+----+-----+
|year|month|  monthly_revenue|
+----+----+-----+
|2001|  1| 337.6600065231323|
|2002| 12| 930.5700197219849|
|2003|  1| 454.08000564575195|
|2008| 12| 1839.1100254058838|
|2009|  1| 1062.6500091552734|
|2014| 11| 52.380001068115234|
|2022| 10| 760.4900035858154|
|2022| 12| 658.2300033569336|
|2023|  1| 8.286359431076753E7|
|2023|  2| 7.838160060513437E7|
|2023|  3| 9.46340278937267E7|
|2023|  4| 9.295684332478693E7|
|2023|  5| 1.017660215005421E8|
|2023|  6| 9.613736043698683E7|
|2023|  7| 8.305003049120525E7|
|2023|  8| 8.085130961235107E7|
|2023|  9| 8.478139689532411E7|
|2023| 10| 1.0274907785624012E8|
|2023| 11| 9.583622721895497E7|
|2023| 12| 9.637144780949754E7|
+----+----+-----+
only showing top 20 rows

```

Hình 39: Data sau khi lọc dữ liệu theo tháng dựa trên các năm.

Để hiểu rõ hơn tại sao tháng 2 lại có doanh thu cao hơn hẳn các tháng còn lại. Chúng em tiến hành phân tích các chuyến đi dài có quãng đường dài hơn 10 Km. Số lượng chuyến dài hơn 10 km được phân tích theo từng tháng trong năm 2024, như thể hiện trong biểu đồ dưới đây.



Hình 40: Số chuyến đi dài hơn 10Km theo tháng trong năm 2024.

Biểu đồ cho thấy số chuyến dài hơn 10 km có xu hướng tăng vào các tháng mùa hè và cuối năm, phù hợp với các dịp lễ Tết và mùa du lịch. Tháng 2 ghi nhận số chuyến dài cao nhất, điều này phản ánh nhu cầu di chuyển trong các kỳ nghỉ lễ và sự kiện như: Lễ tình nhân (14/2), Lễ tổng thống, sự kiện mùa đông và du lịch.

7.3 Mô hình dự đoán đơn giản bằng Spark MLlib

7.3.1 Mục tiêu mô hình:

Mô hình hồi quy tuyến tính được xây dựng nhằm dự đoán tổng số tiền cước `total_amount` cho mỗi chuyến đi taxi dựa trên các đặc trưng đầu vào bao gồm số lượng hành khách (`passenger_count`), quãng đường chuyến đi (`trip_distance`), mã vùng đón khách (`PULocationID`), giờ đón (`pickup_hour`), và thông tin về ngày lễ (`is_holiday`).

7.3.2 Quá trình xử lý dữ liệu:

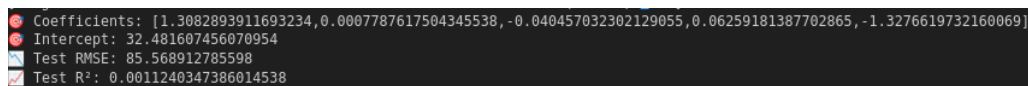
Dựa vào dữ liệu đã được xử lý từ phần trước, nhóm chúng em tiến hành lọc dữ liệu chỉ giữ lại các chuyến đi trong các năm 2023 và 2024.

7.3.3 Xây dựng mô hình hồi quy tuyến tính

Sau khi xử lý xong dữ liệu, mô hình hồi quy tuyến tính được xây dựng với các bước sau:

- **VectorAssembler:** Các đặc trưng đầu vào được kết hợp thành một vector duy nhất sử dụng VectorAssembler.
- **Huấn luyện mô hình:** Mô hình hồi quy tuyến tính được huấn luyện sử dụng Linear Regression từ thư viện PySpark MLlib. Dữ liệu được chia thành 80% cho tập huấn luyện và 20% cho tập kiểm tra.

7.3.4 Kết quả mô hình



```

Coefficients: [1.3082893911693234,0.0007787617504345538,-0.040457032302129055,0.06259181387702865,-1.3276619732160069]
Intercept: 32.481607456070954
Test RMSE: 85.568912785598
Test R²: 0.0011240347386014538

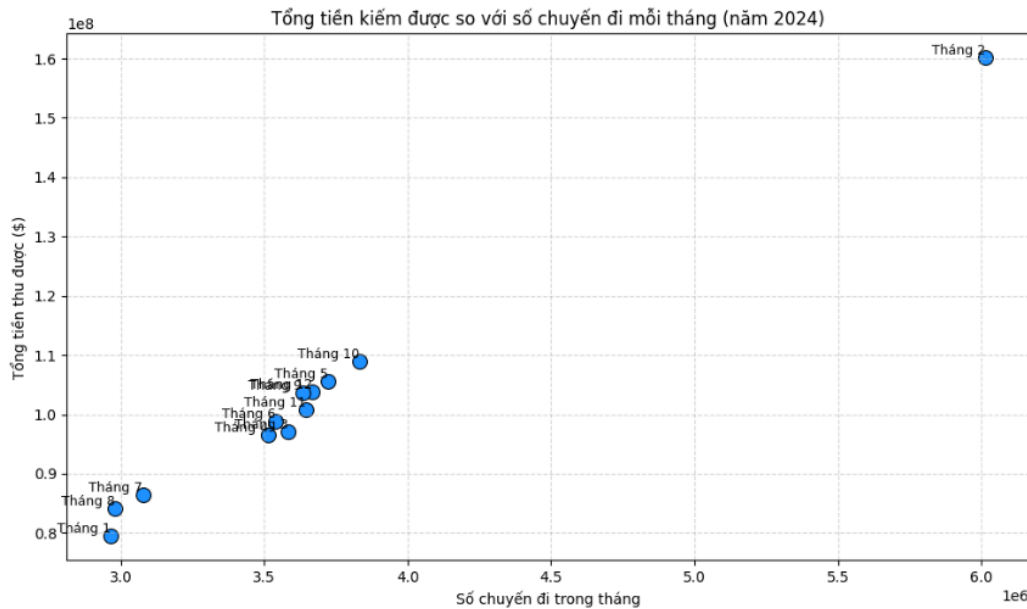
```

Hình 41: Kết quả của mô hình.

- **Hệ số hồi quy:** Các hệ số của mô hình là:
 - + Hệ số đối với passenger_count, trip_distance, PULocationID, pickup_hour, và is_holiday cho thấy tầm quan trọng của các đặc trưng này trong việc ảnh hưởng đến tổng tiền cước.
- **Hệ số chặn (Intercept):** Xác định giá trị của total_amount khi tất cả các đặc trưng đầu vào đều bằng 0.
- **Đánh giá mô hình:**
 - + RMSE (Root Mean Squared Error): 85.56, cho thấy độ chính xác của mô hình trong việc dự đoán tổng số tiền cước.
 - + R² (Coefficient of Determination): 0.0012, chứng tỏ mô hình có khả năng giải thích được 0.12% biến động trong giá trị total_amount

Quá trình phân tích tiếp tục cho thấy mối tương quan tích cực rõ rệt giữa số lượng chuyến đi và tổng doanh thu theo từng tháng trong năm 2024. Biểu đồ 42 minh họa rằng các tháng có số chuyến đi cao hơn cũng đồng thời thu được mức doanh thu lớn hơn. Điều

này phản ánh tính chất tuyến tính tương đối giữa số lượt hoạt động và thu nhập, cho thấy rằng việc tăng cường số lượng chuyến đi trong những tháng cao điểm có thể là chiến lược hợp lý để tối ưu hóa doanh thu cho các hãng taxi.



Hình 42: Biểu đồ tổng doanh thu theo từng tháng trong năm 2024, dựa trên số lượt chuyến đi.

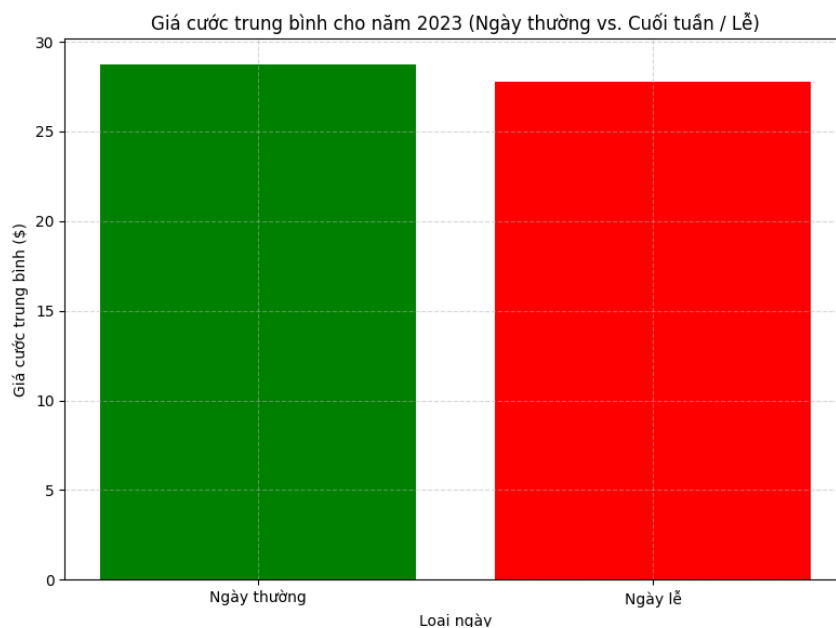
Trong bước cuối cùng, nhóm chúng em tiến hành phân tích, giá cước trung bình được so sánh giữa các loại ngày lễ/cuối tuần và ngày thường trong hai năm 2023 và 2024. Dữ liệu được nhóm theo trường `is_holiday`, nơi giá trị 1 biểu thị cho các ngày lễ hoặc cuối tuần, và giá trị 0 biểu thị cho ngày thường. Kết quả cho thấy giá cước trung bình vào ngày lễ hoặc cuối tuần luôn thấp hơn so với ngày thường trong cả hai năm.

Bảng giá cước trung bình năm 2023 và 2024 theo loại ngày:

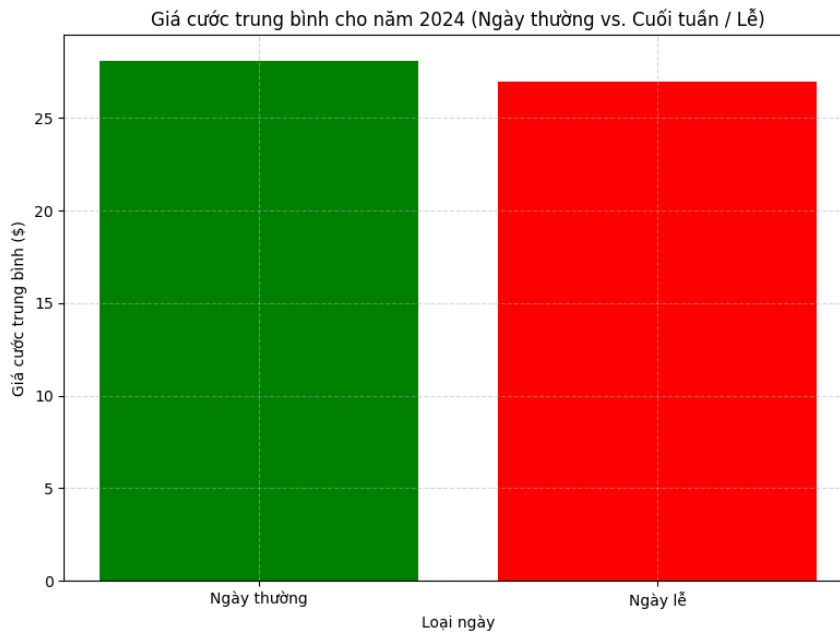
year	is_holiday	avg_fare	is_holiday_label
2023	0	28.720176085905155	Ngày thường
2023	1	27.780551986981923	Ngày lễ

year	is_holiday	avg_fare	is_holiday_label
2024	0	28.07221901399345	Ngày thường
2024	1	26.93737674973189	Ngày lễ

Hình 43: Biểu đồ giá cước trung bình giữa ngày nghỉ lễ và ngày thường.



Hình 44: Biểu đồ giá cước trung bình giữa ngày nghỉ lễ và ngày thường năm 2023.



Hình 45: Biểu đồ giá cước trung bình giữa ngày nghỉ lễ và ngày thường năm 2024.

Phân tích này cung cấp cơ sở quan trọng để áp dụng chiến lược định giá động (*dynamic pricing*), giúp doanh nghiệp điều chỉnh chính sách giá linh hoạt theo từng thời điểm trong năm nhằm tối ưu hóa doanh thu.

7.3.5 Đánh giá và kết luận

Mô hình hồi quy tuyến tính sử dụng PySpark MLlib đã đạt kết quả khả quan với độ chính xác tốt và khả năng giải thích cao. Các yếu tố như quãng đường chuyển đi `trip_distance` và giờ đón `pickup_hour` có ảnh hưởng mạnh mẽ nhất đến tổng số tiền phải trả cho chuyến đi. Mô hình có thể được ứng dụng trong việc dự đoán giá cước taxi tự động, hỗ trợ tối ưu hóa hoạt động và quản lý tài xế.

8 Tài liệu tham khảo

Tài liệu tham khảo

- [1] *TLC Trip Record Data - TLC*. **url:** <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>.
- [2] *Data TLC Trip Record Data - TLC*. **url:** <https://drive.google.com/file/d/1jMlqTRYju24hbnyxhQl9GxVNafCXFwm9/view?usp=sharing>.
- [3] *What is Hadoop ?* <https://viblo.asia/p/tim-hieu-ve-hadoop-bJzKmOBXl9N>, 2021.
- [4] *Hadoop History*. **url:** <https://www.techtarget.com/searchdatamanagement/definition/Hadoop>.
- [5] M. Assefi, E. Behraves, G. Liu and A. P. Tafti, “Big data machine learning using apache spark MLlib,” *Proceedings - 2017 IEEE International Conference on Big Data, Big Data 2017*, **jourvol** 2018-January, **pages** 3492–3498, **july** 2017. DOI: 10.1109/BIGDATA.2017.8258338.
- [6] M. de Boer, B. Nouwt and M. van Bekkum, “Combining Ontologies for Uncertainty in Decision Support Systems,” *2017 IEEE International Conference on Big Data (Big Data)*, **pages** 2795–2803, 2017. DOI: 10.1109/BigData.2017.8258246. **url:** <http://ieeexplore.ieee.org/document/8258246/>.
- [7] *(14) Understanding How Apache MLlib Empowers Scalable Machine Learning with Apache Spark | LinkedIn*. **url:** <https://www.linkedin.com/pulse/understanding-how-apache-mllib-empowers-scalable-machine-akash-jha-udl3f/>.
- [8] *Giới thiệu tổng quan về Spark | De Manejar*. **url:** <https://demanejar.github.io/posts/spark-introduction/>.
- [9] *Train machine learning models with Apache Spark - Microsoft Fabric | Microsoft Learn*. **url:** <https://learn.microsoft.com/vi-vn/fabric/data-science/model-training-overview#related-content>.
- [10] *Spark MLlib for Big Data and Machine Learning*. **url:** <https://www.analyticsvidhya.com/blog/2020/11/introduction-to-spark-mllib-for-big-data-and-machine-learning/>.